Índice

Endpoint 1 — Iniciar sesión 11

1. Para qué sirve 11

2. Autenticación 11

3. Encabezados http 11

4. Cuerpo de la solicitud 11

5. Funcionamiento del endpoint 12

6. Respuesta exitosa — 200 OK 13

7. Respuestas de error13

8. Relación con la historia de usuario 14

9. Validaciones 14

10. Reglas de negocio 14

11. Relación con la base de datos 15

12. Relaciones principales 15

Endpoint 2 — Consultar usuario autenticado 17

1. Para qué sirve 17

2. Autenticación 17

3. Encabezados HTTP 17

4. Respuesta exitosa — 200 OK 18

5. Respuestas de error18

6. Relación con la historia de usuario 18

7. Validaciones 18

8. Reglas de negocio 19

9. Relación con la base de datos 19

Relaciones principales 19

Endpoint 3 — Cerrar sesión 20

6. Validaciones 21

Endpoint 4 — Solicitar recuperación de contraseña 22

2. Para qué sirve? 23

3. Autenticación 23

9. Validaciones 24

1. Reglas de negocio 25

10. Relación con la base de datos 25

Endpoint 5 — Restablecer contraseña 25

2. Para qué sirve? 25

Permite que un usuario establezca una contraseña nueva cuando ha solicitado recuperar el acceso a su cuenta. 25

El sistema debe comprobar que el token de recuperación sea válido, que no haya expirado y que no se haya utilizado anteriormente. 25

Después de actualizar la contraseña, el sistema debe restablecer los datos de bloqueo de la cuenta y revocar las sesiones activas para garantizar que la contraseña anterior no siga permitiendo el acceso. 26

3. Autenticación 26

8. Validaciones 27

Reglas de negocio 27

9. Relación con la base de datos 28

Endpoint 6 - iniciar sesion 28

2. ¿Para qué sirve? 29

3. Autenticación 29

4. Encabezados de la solicitud 29

5. Cuerpo de la solicitud 29

6. Respuesta exitosa 30

Validaciones 31

10. Reglas de negocio 32

11. Seguridad y gestión del JWT 32

Se propone utilizar JSON Web Token (JWT) como mecanismo de autenticación para los endpoints protegidos. 32

El token debería incluir: 32

sub: identificador UUID de la cuenta. 32

jti: identificador único del token o sesión. 32

iat: fecha de emisión. 32

exp: fecha de expiración. 33

No se deben incluir contraseñas, hashes de contraseñas ni información sensible. 33

Registro de sesiones 33

Se propone almacenar el hash del token en session.token_hash, en lugar de guardar el JWT original. 33

El backend podrá validar el JWT y comprobar que la sesión asociada continúe activa. 33

Esto permitirá gestionar la revocación de sesiones y evitar que un token revocado siga siendo aceptado. 33

La estructura definitiva dependerá de las columnas y restricciones de la tabla session. 33

9. Relación con la base de datos 33

Endpoint 7 — Listar vehículos 33

2. ¿Para qué sirve? 34

3. Autenticación y permisos 34

4. Encabezados http 34

5. Parámetros de consulta 34

6. Respuestas de éxito34

7. Validaciones del backend 34

8. Reglas de negocio 35

9. Relación con la base de datos 35

10. Relación con las historias de usuario 35

Endpoint 8 — Consultar vehículo 35

Endpoint 9 — Actualizar información del vehículo 39

Endpoint 10 — Cambiar estado del vehículo 43

Endpoint 11 — Consultar tipos de vehículo 48

Endpoint 12 — Consultar estados de vehículo 51

11. Relación con las historias de usuario 54

Endpoint 13 — Listar personal 55

1. ¿Para qué sirve? 55

2. Autenticación 55

3. Encabezados: HTTP 55

4. Parámetros de consulta 55

5. Respuesta exitosa 56

7. Validaciones 57

8. Reglas de negocio 58

9. Relación con la base de datos 58

10. Relación con las historias de usuario 58

Endpoint 14 — Consultar integrante del personal 59

2. ¿Para qué sirve? 59

3. Autenticación 59

5. Parámetros de ruta 59

6. Respuesta exitosa 60

8. Validaciones 60

9. Reglas de negocio 61

10. Relación con la base de datos 61

Endpoint 15 — Registrar integrante del personal 62

1. ¿Para qué sirve? 62

2. Autenticación 62

3. Cuerpo de la solicitud 63

6. Validaciones 64

7. Reglas de negocio 65

9. Relación con la base de datos 65

10. Relación con las historias de usuario 65

Endpoint 16 — Actualizar información del integrante del personal 66

1. ¿Para qué sirve? 66

2. Autenticación 66

5. Respuesta exitosa 67

7. Validaciones 68

8. Reglas de negocio 69

9. Relación con la base de datos 69

10. Relación con las historias de usuario 69

Endpoint 17. Registrar un conductor 70

1. ¿Para qué sirve? 70

7. Validaciones 72

8. Reglas de negocio 72

9. Relación con la base de datos 73

10. Relación con las historias de usuario 73

Endpoint 18/ listar conductores 74

1. ¿Para qué sirve? 74

2. Tablas relacionadas74

3. Parámetros de entrada 74

8. Validaciones 76

9. Reglas de negocio 77

10. Relación con la base de datos 77

11. Relación con las historias de usuario 78

Endpoint 19/ consultar el detalle del conductor 78

1. Para qué sirve? 78

8. Validaciones 80

9. Reglas de negocio 80

10. Relación con la base de datos 81

11. Relación con las historias de usuario 81

Endpoint 20/Actualizar la informacion de un conductor 82

1. ¿Para qué sirve? 82

8. Validaciones 84

9. Reglas de negocio 84

10. Relación con la base de datos 85

11. Relación con las historias de usuario 85

Endpoint 21/ Crear una cuadrilla 86

1. ¿Para qué sirve? 86

6. Validaciones 87

7. Reglas de negocio 87

8. Relación con la base de datos 88

9. Relación con las historias de usuario 88

Endpoint 22/listar cuadrillas 88

1. ¿Para qué sirve? 88

2. Parámetros de entrada 89

3. Respuesta exitosa :http 200 ok 89

5. Validaciones 90

6. Reglas de negocio 90

7. Relación con la base de datos 91

8. Relación con las historias de usuario 91

Endpoint 23/consultar cuadrilla por id 91

1. ¿Para qué sirve? 91

2. Parámetros de entrada 91

6. Validaciones 93

7. Reglas de negocio 93

8. Relación con la base de datos 94

9. Relación con las historias de usuario 94

Endpoint 24/ Actualizar cuadrilla 95

1. ¿Para qué sirve? 95

2. Parámetros de entrada 95

3. Respuesta exitosa:http 200 ok 95

4. Validaciones 96

5. Reglas de negocio 97

6. Relación con la base de datos 97

7. Relación con las historias de usuario 98

Endpoint 25/crear turno 98

1. ¿Para qué sirve? 98

2. Parámetros de entrada 98

3. Respuesta exitosa:http 201 99

4. Respuestas de error99

5. Validaciones 99

6. Reglas de negocio 100

7. Relación con la base de datos 100

8. Relación con las historias de usuario 101

Endpoint 26/listar turno 101

1. ¿Para qué sirve? 101

2. Parámetros de entrada 101

3. Respuesta exitosa : http 200 ok 102

5. Validaciones 103

6. Reglas de negocio 104

7. Relación con la base de datos 104

8. Relación con las historias de usuario 104

Endpoint 27/Consultar turno por id 105

1. ¿Para qué sirve? 105

2. Tablas relacionadas105

3. Parámetros de entrada 105

6. Validaciones 106

7. Reglas de negocio 106

8. Relación con la base de datos 107

9. Relación con las historias de usuario 107

Endpoint 28/actualizar turno 107

1. ¿Para qué sirve? 107

2. Tablas relacionadas108

3. Parámetros de entrada 108

4. Respuesta exitosa: http 200 ok 109

6. Validaciones 109

7. Reglas de negocio 110

8. Relación con la base de datos 110

9. Relación con las historias de usuario 111

Endpoint 29/Listar macrorrutas 111

2. Para qué sirve? 111

3. Autenticación 111

5. Parámetros de consulta 112

6. Respuesta exitosa 113

7. Respuestas de error113

8. Validaciones 114

9. Reglas de negocio 114

10. Relación con la base de datos 114

11. Relación con las historias de usuario 115

Endpoint 30/Crear macrorruta 115

2. Para qué sirve? 115

3. Autenticación 115

6. Respuesta exitosa 117

7. Respuestas de error117

8. Validaciones 118

9. Reglas de negocio 118

10. Relación con la base de datos 118

11. Relación con las historias de usuario 119

endpoint 31/Consultar macrorruta 119

2. Para qué sirve? 119

3. Autenticación 119

4. Encabezados HTTP 119

5. Parámetros de ruta 120

6. Respuesta exitosa 120

7. Respuestas de error121

8. Validaciones 121

9. Reglas de negocio 121

10. Relación con la base de datos 122

11. Relación con las historias de usuario 122

Endpoint 32/Listar microrrutas 122

2. Para qué sirve? 122

3. Autenticación 123

4. Encabezados HTTP 123

5. Parámetros de consulta 123

6. Respuesta exitosa 124

7. Respuestas de error125

8. Validaciones 125

9. Reglas de negocio 126

10. Relación con la base de datos 126

11. Relación con las historias de usuario 126

endpoint 33/Crear microrruta 126

2. Para qué sirve? 127

3. Autenticación 127

4. Encabezados HTTP 127

6. Respuesta exitosa 129

7. Respuestas de error129

8. Validaciones 130

9. Reglas de negocio 130

10. Relación con la base de datos 131

11. Relación con las historias de usuario 131

Endpoint 34/Consultar microrruta 131

2. Para qué sirve? 132

3. Autenticación 132

4. Encabezados HTTP 132

6. Respuesta exitosa 133

7. Respuestas de error134

8. Validaciones 134

9. Reglas de negocio 134

10. Relación con la base de datos 135

11. Relación con las historias de usuario 135

Endpoint 34/Consultar barrios 135

1. Relación con las historias de usuario 137

Endpoint 35/Asociar barrio a microrruta 138

Asignaciones 141

Endpoint 36/Consultar asignaciones 141

Endpoint 37/Consultar una asignación 144

Endpoint 38/Crear asignación 148

Endpoint 39/Modificar asignación 152

Endpoint 40/Consultar historial de una asignación 157

Endpoint 41/Listar turnos de una microrruta 161

Endpoint 42/Asociar turno a una microrruta 163

Endpoint 43/Actualizar turno programado de una microrruta 165

Organización de los endpoints de VINTRA

A partir de las HU1–HU12 que recuperamos, podemos plantear este primer inventario. Las historias contemplan acceso por rol, CRUD de vehículos, estados, personal, cuadrillas, turnos, disponibilidad, macrorrutas, microrrutas, barrios, asignaciones, segundo recorrido y trazabilidad.

Propuesta inicial

Los siguientes endpoints

son un diseño para revisión, no rutas que ya existan en el backend

Endpoint 1 — Iniciar sesión

Endpoint 2 — Consultar usuario autenticado

Endpoint 3 — Cerrar sesión

Endpoint 4 — Solicitar recuperación de contraseña

Endpoint 5 — Restablecer contraseña

Endpoint 6— Listar vehículos

Endpoint 7 — Consultar vehículo

Endpoint 8 — Actualizar información del vehículo

Endpoint 9 — Cambiar estado del vehículo

Endpoint 10 — Consultar tipos de vehículo

Endpoint 11 — Consultar estados de vehículo

Endpoint 12 — Listar personal

Endpoint 13 — Consultar integrante del personal

Endpoint 14 — Registrar integrante del personal

Endpoint 15 — Actualizar información del integrante del personal

Endpoint 16. Registrar un conductor

Endpoint 17/ listar conductores

Endpoint 18/ consultar el detalle del conductor

Endpoint 19/Actualizar la informacion de un conductor

Endpoint 20/ Crear una cuadrilla

Endpoint 21/listar cuadrillas

Endpoint 22/consultar cuadrilla por id

Endpoint 23/ Actualizar cuadrilla

Endpoint 24/crear turno

Endpoint 25/listar turno

Endpoint 26/Consultar turno por id

Endpoint 27/actualizar turno

Endpoint 28/Listar macrorrutas

Endpoint 29/Crear macrorruta

endpoint 30/Consultar macrorruta

Endpoint 31/Listar microrrutas

endpoint 32/Crear microrruta

Endpoint 33/Consultar microrruta

Endpoint 34/Consultar barrios

Endpoint 35/Asociar barrio a microrruta

Endpoint 36/Consultar asignaciones

Endpoint 37/Consultar una asignación

Endpoint 38/Crear asignación

Endpoint 39/Modificar asignación

Endpoint 40/Consultar historial de una asignación

Endpoint 41/Listar turnos de una microrruta

Endpoint 42/Asociar turno a una microrruta

Endpoint 43/Actualizar turno programado de una microrruta

Endpoint 1- iniciar sesion

metodo y ruta: post /api/v1/auth/login

2. ¿Para qué sirve?

Permite que un usuario acceda a VINTRA mediante su correo electrónico y contraseña.

El backend recibe las credenciales, identifica la cuenta asociada al correo electrónico, verifica la contraseña y comprueba que la cuenta tenga permitido iniciar sesión.

Si las credenciales son correctas y la cuenta está habilitada, el sistema genera un token de acceso JWT, registra la sesión correspondiente y devuelve la información necesaria para que el usuario pueda acceder a los recursos autorizados según su rol.

3. Autenticación

Este endpoint no requiere autenticación previa, ya que su finalidad es precisamente permitir que el usuario inicie sesión.

El usuario debe proporcionar:

Correo electrónico registrado en VINTRA.

Contraseña correspondiente a su cuenta.

El backend será responsable de validar las credenciales y generar el token de acceso.

4. Encabezados de la solicitud

Content-Type: application/json

Accept: application/json

5. Cuerpo de la solicitud

El endpoint recibe un objeto JSON con las credenciales del usuario.

```
{
  "email": "usuario@vintra.com",
  "password": "MiContrasena123!"
}
```

campo: email

tipo:string

obligatorio:si

descripcion:Correo electrónico registrado en VINTRA.

campo: password

tipo:string

obligatorio:si

descripcion:Contraseña correspondiente a la cuenta del usuario

6. Respuesta exitosa

Código HTTP: 200 OK

ejemplo de respuesta propuesta


```
{
  "accessToken": "jwt-token-generado",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "user": {
    "id": "uuid-de-la-cuenta",
    "email": "usuario@vintra.com",
    "firstName": "Juan",
    "lastName": "Pérez",
    "role": {
      "code": "ADMIN",
      "name": "Administrador"
    },
    "status": {
      "code": "ACTIVE",
      "name": "Activo"
```

```
  }
 }
}
```

consideraciones

accessToken: JWT generado por el backend.

tokenType: tipo de autenticación utilizado.

expiresIn: tiempo de vida del token expresado en segundos.

user.id: identificador UUID de la cuenta.

user.email: correo electrónico del usuario.

role: información del rol asociado al perfil.

status: estado de la cuenta.

respuestas de error

400 Bad Request:Faltan campos obligatorios o el formato del correo no es válido

401 Unauthorized:El correo electrónico o la contraseña son incorrectos.

403 Forbidden:La cuenta existe, pero su estado no permite iniciar sesión.

423 Locked:La cuenta tiene un bloqueo temporal vigente, si se adopta esta respuesta.

429 Too Many Requests: Se superó el límite de solicitudes permitido.

500 Internal Server Error:Ocurrió un error inesperado durante el proceso de autenticación

Validaciones

El backend debe realizar las siguientes validaciones durante el inicio de sesión:

Comprobar que se hayan enviado los campos email y password.

Validar que el correo electrónico tenga un formato válido.

Normalizar el correo electrónico para realizar una búsqueda consistente, sin distinguir entre mayúsculas y minúsculas.

Buscar el correo en la tabla user_data.

Identificar la cuenta correspondiente en user_account mediante la relación entre sus identificadores UUID.

Verificar si la cuenta tiene un bloqueo temporal vigente mediante el campo locked_until.

Consultar el estado de la cuenta mediante status_id y la tabla user_status.

Comparar la contraseña recibida con el hash almacenado en user_account.pwd_hash, utilizando el algoritmo de verificación correspondiente.

Si las credenciales son incorrectas, incrementar failed_attempts y evaluar la política de bloqueo.

Si las credenciales son correctas, reiniciar el contador de intentos fallidos.

Obtener el perfil y el rol asociado a la cuenta.

Actualizar el campo last_login_at con la fecha y hora del inicio de sesión.

Generar un JWT con la información necesaria para identificar al usuario y su sesión.

Registrar la sesión correspondiente en la tabla session.

Devolver el token de acceso y la información del usuario

10. Reglas de negocio

RN 01 Acceso según el rol: el usuario únicamente podrá acceder a las funcionalidades autorizadas para su rol.

RN-02 Integridad de datos: la sesión debe quedar asociada a una cuenta existente.

RN 03 Autenticación mediante credenciales: el acceso debe realizarse mediante correo electrónico y contraseña.

RN 04 Protección de contraseñas: las contraseñas deben almacenarse y verificarse mediante mecanismos de hash seguros. Nunca deben compararse ni almacenarse en texto plano.

RN 05 Estado de la cuenta: únicamente las cuentas habilitadas podrán iniciar sesión.

RN 06 Control de intentos fallidos: el sistema debe registrar los intentos fallidos y aplicar la política de bloqueo temporal definida.

RN 07 Registro de acceso: cuando el inicio de sesión sea exitoso, debe actualizarse last_login_at.

RN 08 Gestión de sesiones: cada inicio de sesión exitoso debe generar un registro asociado a la cuenta correspondiente.

RN 09 Protección de recursos: el token generado debe permitir identificar al usuario y validar su acceso a los endpoints protegidos.

RN 10 Revocación de sesiones: el sistema debe permitir invalidar sesiones cuando corresponda, por ejemplo, durante el cierre de sesión.

11. Seguridad y gestión del JWT

Se propone utilizar JSON Web Token (JWT) como mecanismo de autenticación para los endpoints protegidos.

El token debería incluir:

sub: identificador UUID de la cuenta.

jti: identificador único del token o sesión.

iat: fecha de emisión.

exp: fecha de expiración.

No se deben incluir contraseñas, hashes de contraseñas ni información sensible.

Registro de sesiones

Se propone almacenar el hash del token en session.token_hash, en lugar de guardar el JWT original.

El backend podrá validar el JWT y comprobar que la sesión asociada continúe activa.

Esto permitirá gestionar la revocación de sesiones y evitar que un token revocado siga siendo aceptado.

La estructura definitiva dependerá de las columnas y restricciones de la tabla session**.**

9. Relación con la base de datos

El endpoint utiliza las siguientes tablas del modelo d:e VINTRA

user_data :Consultar el correo electrónico y los datos personales del usuario.

user_account: Consultar el hash de la contraseña, el estado de la cuenta y los datos de control de autenticación.

user_status:Verificar si la cuenta está habilitada para iniciar sesión.

profile:Identificar el perfil asociado a la cuenta.

user_role: Obtener el rol y los permisos asociados al usuario.

session:Registrar la sesión generada durante el inicio de sesión.

Endpoint 2 — Consultar usuario autenticado

Método y ruta: GET /api/v1/auth/me

Autenticación: Bearer Token

Body: No requiere

1. Para qué sirve

Este endpoint permite consultar la información del usuario que ha iniciado sesión en VINTRA.

Su finalidad es que el frontend pueda conocer qué usuario está utilizando el sistema, cuál es el estado de su cuenta y qué rol tiene asignado.

Esta información permitirá mostrar las opciones correspondientes y aplicar las restricciones de acceso según el rol del usuario autenticado.

2. Autenticación

Requiere que el usuario tenga una sesión válida y se envíe el token JWT en el encabezado Authorization.

El backend realizará el siguiente proceso de verificación:

Recibir el token JWT enviado en el encabezado Authorization.

Validar la firma y vigencia del token.

Identificar la sesión correspondiente en la tabla session.

Verificar que la sesión no haya sido revocada (revoked_at IS NULL).

Comprobar que la sesión no haya expirado (expires_at debe ser posterior a la fecha y hora actual).

Identificar la cuenta asociada mediante session.user_account_id.

Consultar la información personal en user_data.

Obtener el estado de la cuenta mediante user_status.

Obtener el perfil activo en profile y consultar su rol en user_role.

3. Encabezados HTTP

Authorization: Bearer <access_token>

Accept: application/json

4. Respuesta exitosa — 200 OK

Estructura basada en las tablas user_account, profile y user_role del modelo de datos:


```
{
  "id": "uuid-del-usuario",
  "documentId": "123456789",
  "firstName": "María",
  "lastName": "Montes",
  "email": "usuario@correo.com",
  "phone": "3001234567",
  "status": {
    "id": "uuid-del-estado",
    "code": "ACTIVE",
    "name": "Activo"
  },
  "role": {
    "id": "uuid-del-rol",
    "code": "ADMIN",
    "name": "Administrador"
  }
}
```


5. Respuestas de error

401 Unauthorized: No existe una sesión válida o no se proporcionó autenticación.

403 Forbidden: El usuario está autenticado, pero no tiene permiso para acceder al recurso.

500 Internal Server Error: Ocurrió un error inesperado en el servidor.

6. Relación con la historia de usuario

HU1 Acceso según rol.

7. Validaciones

Verificar que se haya enviado el token JWT.

Validar la firma y vigencia del token.

Comprobar que corresponda a una sesión existente.

Verificar que expires_at no haya vencido.

Comprobar que revoked_at sea NULL.

Verificar que la cuenta esté activa según el catálogo user_status.

Consultar los datos personales asociados en user_data.

Obtener el perfil cuyo is_active sea TRUE.

Consultar el rol asociado a ese perfil en user_role.

8. Reglas de negocio

RN 01 Acceso según rol: el acceso a las funcionalidades debe corresponder al rol del usuario autenticado.

RN 02 Identificación del usuario: La información del usuario debe obtenerse de la sesión validada, no de un identificador enviado por el frontend.

RN 03 Perfil activo: Solo se debe utilizar el perfil activo del usuario.

RN 04 Estado de la cuenta: Una cuenta inactiva no debe acceder a las funcionalidades protegidas del sistema.

9. Relación con la base de datos

Este endpoint involucra las siguientes tablas:

session: validar la sesión y comprobar su vigencia y revocación.

user_account: obtener la cuenta, el documento y el estado del usuario.

user_data: consultar nombres, apellidos, correo y teléfono.

user_status: obtener el código y nombre del estado de la cuenta.

profile: identificar el perfil activo del usuario.

user_role: obtener el rol asociado al perfil activo.

document_type: si se requiere mostrar el tipo de documento, consultar su catálogo.

Relaciones principales

session.user_account_id → user_account.id

user_data.id → user_account.id

user_account.status_id → user_status.id

user_account.document_type_id → document_type.id

profile.user_account_id → user_account.id

profile.user_role_id → user_role.id

Endpoint 3 — Cerrar sesión

Método y ruta :POST /api/v1/auth/logout

Autenticación : Bearer Token

Body : No requiere

1. ¿Para qué sirve?

Cerrar la sesión actual del usuario autenticado

Este endpoint permite que un usuario cierre su sesión de VINTRA de forma segura.

Cuando el usuario selecciona la opción de cerrar sesión, el frontend realiza una solicitud al backend. Este identifica la sesión actual y la revoca para impedir que el mismo token pueda seguir utilizándose.

El objetivo es finalizar el acceso del usuario y evitar que una sesión cerrada continúe siendo válida.

3. Autenticación

Este endpoint requiere un Bearer Token válido.

El frontend enviará

Authorization: Bearer <access_token>

El backend deberá:

Recibir el token.

Validarlo mediante el mecanismo establecido.

Localizar la sesión correspondiente.

Verificar que no haya expirado ni sido revocada.

Actualizar revoked_at con la fecha y hora actual.

Confirmar el cierre de sesión.

2. Encabezados http

Authorization: Bearer <access_token>

Accept: application/json

3. Funcionamiento

Cuando el usuario solicite cerrar sesión, el backend deberá:

Recibir el token enviado en el encabezado Authorization.

Validar que corresponda a una sesión existente y vigente.

Identificar la sesión asociada al token.

Actualizar el campo revoked_at de esa sesión con la fecha y hora del cierre.

Confirmar que la sesión quedó revocada.

Importante: se revoca únicamente la sesión actual, no todas las sesiones del usuario.

4. Respuesta exitosa 204 No Content

No devuelve un cuerpo de respuesta. El código HTTP indica que la solicitud se procesó correctamente.

5. Posibles errores

401 Unauthorized :Token ausente, inválido, expirado o sesión ya revocada.

500 Internal Server Error : Error inesperado al procesar el cierre de sesión.

5. Relación con la historia de usuario

HU1 Acceso según rol

6. Validaciones

Verificar que se envíe el encabezado Authorization.

Comprobar que el token tenga el formato esperado.

Validar que corresponda a una sesión existente.

Verificar que la sesión no haya expirado.

Comprobar que revoked_at sea NULL.

Actualizar revoked_at con la fecha y hora actual.

Confirmar que la actualización se haya realizado correctamente

10. Reglas de negocio

RN 01 Acceso por rol: el cierre de sesión debe estar disponible para el usuario autenticado, independientemente del rol que tenga asignado.

RN 02 Revocación de sesión: una sesión revocada no debe permitir nuevas solicitudes protegidas con el mismo token.

RN 03 Cierre individual: cerrar una sesión no debe revocar automáticamente las demás sesiones del usuario.

RN 04 Integridad de sesión: solamente se debe actualizar la sesión que corresponde al token presentado.

RN 5 Vigencia: una sesión expirada o previamente revocada no debe tratarse como una sesión activa.

7. Relación con la base de datos

La operación afecta principalmente a la tabla session

id : Identifica el registro de sesión

user_account_id : Identifica la cuenta propietaria de la sesión

token_hash : Identificar la sesión mediante el mecanismo de validación del token.

revoked_at : Registrar cuándo se revocó la sesión.

expires_at : Comprobar la vigencia de la sesión.

Endpoint 4 — Solicitar recuperación de contraseña

metodo y ruta : post /api/v1/auth/forgot-password

Autenticación : No requiere Bearer Token

Body: Sí

2. Para qué sirve?

Este endpoint permite que una persona que ha olvidado su contraseña solicite recuperar el acceso a VINTRA.

El usuario proporciona su correo electrónico y el backend busca la cuenta asociada. Si cumple las condiciones para iniciar la recuperación, el sistema genera un token temporal y envía las instrucciones al correo registrado.

El objetivo es permitir que el usuario pueda recuperar el acceso sin necesidad de iniciar sesión.

3. Autenticación

Este endpoint no requiere Bearer Token porque está dirigido a usuarios que no tienen acceso a su cuenta.

Sin embargo, debe implementar controles de seguridad para evitar solicitudes abusivas y proteger la información de los usuarios.

4. Encabezados: http

Content-Type: application/json

Accept: application/json

Cuerpo de solicitud — Request Body

```
{
  "email": "usuario@correo.com"
}
```

campo : email

tipo: string

descripcion : correo registrado en VINTRA

6. Funcionamiento del endpoint

El backend deberá realizar el siguiente proceso:

Recibir el correo electrónico.

Validar que el campo esté presente y tenga un formato válido.

Buscar el correo en la tabla user_data.

Identificar la cuenta asociada mediante la relación correspondiente.

Verificar que la cuenta cumpla las condiciones para recuperar la contraseña.

Generar un token de recuperación seguro y aleatorio.

Calcular el hash del token.

Registrar la solicitud en password_reset_token.

Establecer la fecha de expiración del token.

Enviar las instrucciones de recuperación al correo electrónico registrado.

7. Respuesta exitosa — 202 Accepted

{

"message": "Si el correo está registrado y la cuenta cumple las condiciones de recuperación, recibirás instrucciones para continuar."

}

Respuestas de error

400 Bad Request : El correo no fue proporcionado o no tiene un formato válido.

429 Too Many Requests : Se superó el límite permitido de solicitudes de recuperación.

500 Internal Server Error : Ocurrió un error inesperado durante el procesamiento.

9. Validaciones

Verificar que el correo electrónico esté presente.

Comprobar que tenga un formato válido.

Buscar el correo en user_data.

Identificar correctamente la cuenta asociada.

Verificar que la cuenta cumpla las condiciones de recuperación.

Generar un token seguro y temporal.

Almacenar únicamente el hash del token.

Establecer una fecha de expiración.

Aplicar límites a las solicitudes.

Evitar revelar la existencia de cuentas mediante los mensajes de respuesta.

1. Reglas de negocio

RN 01 Seguridad de recuperación: el proceso debe utilizar tokens seguros y temporales.

RN 02 Uso único: cada token de recuperación solamente puede utilizarse una vez.

RN 03 Expiración: un token vencido no debe permitir continuar con la recuperación.

RN 04 Protección de cuentas: el sistema no debe revelar si un correo está registrado.

RN 05 Integridad de datos RN-08: cada solicitud de recuperación debe estar relacionada con una cuenta existente.

RN 06 Protección contra abusos: deben establecerse límites para evitar solicitudes excesivas.

RN 07 Entrega segura: las instrucciones de recuperación deben enviarse al correo registrado en la cuenta.

10. Relación con la base de datos

Este endpoint utiliza las siguientes tablas:

user_data : Permite consultar el correo electrónico registrado.

user_account : Permite identificar la cuenta asociada y consultar su estado.

user_status: Permite verificar el estado de la cuenta.

password_reset_token : Almacena el hash del token, la cuenta asociada, la IP de solicitud, la expiración y la utilización del token.

Endpoint 5 — Restablecer contraseña

metodo y ruta : POST /api/v1/auth/reset-password

2. Para qué sirve?

Permite que un usuario establezca una contraseña nueva cuando ha solicitado recuperar el acceso a su cuenta.

El sistema debe comprobar que el token de recuperación sea válido, que no haya expirado y que no se haya utilizado anteriormente.

Después de actualizar la contraseña, el sistema debe restablecer los datos de bloqueo de la cuenta y revocar las sesiones activas para garantizar que la contraseña anterior no siga permitiendo el acceso.

3. Autenticación

No requiere una sesión activa ni un Bearer Token, porque está pensado para usuarios que no pueden iniciar sesión.

En su lugar, la solicitud debe incluir el token temporal de recuperación.

El backend deberá verificar que el token corresponda a una solicitud de recuperación válida y que esté asociado a una cuenta existente

4. Encabezados : http

Content-Type: application/json

Accept: application/json

5. Cuerpo de la solicitud :json

```
{
  "token": "token-de-recuperacion",
  "newPassword": "NuevaClaveSegura123!",
  "confirmPassword": "NuevaClaveSegura123!"
}
```

campo: token

tipo: string

obligatorio: si

descripcion:oken temporal recibido para recuperar la cuenta.

campo:newPassword

tipo: string

obligatorio : si

descripcion: Nueva contraseña que desea establecer el usuario.

campo: confirmPassword

tipo: string

obligatorio: si

obligatorio : si

descripcion: Confirmación de la nueva contraseña. Debe coincidir con newPassword.

6. Respuesta exitosa : Código HTTP: 204

No se devuelve un cuerpo de respuesta. El código indica que la contraseña se actualizó correctamente.

7. Respuestas de error :

400 Bad Request : Faltan campos o la nueva contraseña no cumple la política establecida.

401 Unauthorized : El token es inválido, expiró o ya fue utilizado.

500 Internal Server Error : Ocurrió un error inesperado durante el proceso.

8. Validaciones

Antes de actualizar la contraseña, el backend debe:

Verificar que se hayan enviado el token y los campos de contraseña.

Comprobar que las contraseñas coincidan.

Verificar que el token corresponda a un registro de recuperación válido.

Comprobar que el token no haya expirado.

Comprobar que el token no haya sido utilizado anteriormente.

Identificar la cuenta asociada al token mediante user_account_id.

Validar que la nueva contraseña cumpla la política de seguridad establecida.

Generar el hash de la nueva contraseña.

Actualizar pwd_hash en user_account.

Actualizar password_changed_at y updated_at.

Restablecer failed_attempts a 0.

Establecer locked_until en NULL.

Marcar el token de recuperación como utilizado.

Revocar las sesiones activas asociadas a la cuenta.

.

Reglas de negocio

RN 01 Seguridad de recuperación: la recuperación debe realizarse mediante un token temporal y verificable.

RN 02 Integridad de datos: el token debe estar asociado a una cuenta existente.

RN 03 Uso único: el token solamente puede utilizarse una vez.

RN 04 Vigencia del token: el token tendrá una vigencia de 30 minutos.

RN 05 Protección contra reutilización: un token vencido o utilizado anteriormente no puede permitir otro cambio de contraseña.

RN 06 Protección de contraseñas: la contraseña debe almacenarse como hash, nunca como texto plano.

RN 07 Política de contraseñas: la nueva contraseña deberá cumplir la política de seguridad establecida.

RN 08 Revocación de sesiones: después de cambiar la contraseña, se deben revocar todas las sesiones activas de esa cuenta.

RN 09 Notificación de seguridad: después del cambio, se enviará una notificación al correo electrónico registrado.

RN 10 Integridad de la operación: la actualización de la contraseña, el uso del token y la revocación de sesiones se realizarán dentro de una misma transacción de base de datos.

9. Relación con la base de datos

Este endpoint involucra principalmente tres tablas:

password_reset_token :Verificar el token, su vencimiento y si ya fue utilizado.

user_account : Actualizar pwd_hash, password_changed_at, updated_at, failed_attempts y locked_until.

session :Revocar las sesiones activas de la cuenta.

La relación principal es:

password_reset_token.user_account_id → user_account.id

Y las sesiones se relacionan mediante:

session.user_account_id → user_account.id

Vehículos

Endpoint 6— Listar vehículos

1. Información general

Método y ruta: GET /api/v1/vehicles

Autenticación: Bearer Token (JWT)

Body: No requiere.

2. ¿Para qué sirve?

Permite consultar los vehículos registrados en la flota para que el jefe de operaciones pueda conocer sus características, capacidad, identificación y estado actual.

El sistema debe mostrar un listado de vehículos y permitir aplicar filtros para facilitar la búsqueda y consulta de la información.

3. Autenticación y permisos

Requiere una sesión activa y un Bearer Token válido.

El sistema debe identificar al usuario autenticado y verificar que tenga permisos para consultar la información de la flota, de acuerdo con su rol.

4. Encabezados http

Authorization: Bearer <token>

Content-Type: application/json

Accept: application/json

5. Parámetros de consulta

No requiere un cuerpo de solicitud Los criterios de filtrado se enviarán mediante parámetros en la URL

Ejemplo con filtros:

GET /api/v1/vehicles?status=available&type=truck

6. Respuestas de éxito

Respuesta exitosa:

200 OK: Devuelve el listado de vehículos registrados con su información principal.

Respuestas de error:

401 Unauthorized: No se proporcionó un token válido o la sesión no está activa.

403 Forbidden: El usuario no tiene permisos para consultar la flota.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

7. Validaciones del backend

Antes de devolver la información, el backend debe realizar el siguiente proceso:

Verificar que el usuario esté autenticado.

Validar que la sesión asociada al token esté vigente y no haya sido revocada.

Comprobar que el rol del usuario tenga permiso para consultar los vehículos.

Consultar los vehículos registrados en la tabla vehicle.

Obtener la información relacionada con el tipo y estado del vehículo mediante sus respectivas relaciones.

Validar los parámetros de filtrado cuando sean proporcionados.

Devolver los resultados correspondientes a los criterios de búsqueda.

8. Reglas de negocio

RN 01 Acceso según rol: Solamente los usuarios autorizados podrán consultar la información de la flota.

RN 02 Integridad de datos: La información consultada debe corresponder a vehículos registrados y relacionados correctamente con sus catálogos.

RN 03 Información de vehículos: El listado debe permitir identificar las características principales de cada vehículo.

RN 04 Estado del vehículo: El estado actual del vehículo debe estar disponible para su consulta.

RN 05 Filtrado de información: Los filtros deben utilizar criterios definidos por el módulo de flota.

RN 06 Identificación de vehículos inhabilitados: Los vehículos inhabilitados deben identificarse claramente como no disponibles (HU3).

9. Relación con la base de datos

Este endpoint involucra principalmente tres tablas:

vehicle: Contiene la información general del vehículo (placa, capacidad, estado activo).

vehicle_type: Permite obtener el tipo de vehículo asociado.

vehicle_status: Permite consultar el estado operativo actual.

10. Relación con las historias de usuario

HU2 Gestionar vehículos:

Consultar los vehículos registrados.

Identificar la capacidad y características principales.

Aplicar filtros para facilitar la consulta de la flota.

HU3 Gestionar estado de vehículos:

Mostrar el estado actual del vehículo.

Identificar claramente los vehículos inhabilitados como no disponibles.

Endpoint 7— Consultar vehículo

metodo y ruta : GET /api/v1/vehicles/{id}

autenticacion :bear token

body:no requiere

Para qué sirve?

Permite consultar el detalle de un vehículo registrado en la flota: placa, tipo, estado actual y capacidades.

Se usa para mostrar la ficha del vehículo y para precargar el formulario de edición.

3.autenticacion:

Requiere una sesión activa y un Bearer Token válido.

El sistema debe identificar al usuario autenticado y verificar que su rol tenga permisos para consultar la información de la flota, de acuerdo con la matriz de roles y permisos.

Encabezados : http

Authorization: Bearer <access_token>

Accept: application/json

5.Parámetros de ruta No requiere cuerpo de solicitud.

El identificador del vehículo se envía directamente en la URL.

campo: id

tipo: string (UUID)

obligatorio: si

descripcion: Identificador del vehículo que se desea consultar.

6.Respuesta exitosa : Código HTTP: 200 OK

Devuelve la información detallada del vehículo consultado.

{

"id": "uuid-vehiculo",

```
  "plate": "ABC123",
  "vehicleType": {
    "id": "uuid-tipo",
    "code": "COMPACTADOR",
    "name": "Compactador"
  },
  "vehicleStatus": {
    "id": "uuid-estado",
    "code": "AVAILABLE",
    "name": "Disponible"
  },
  "capacityKg": 5000,
  "capacityYd3": 7,
  "isActive": true,
  "createdAt": "2026-11-02T10:00:00Z",
  "updatedAt": "2026-11-02T10:00:00Z"
}
```


7. posibles errores :

400 Bad Request : El identificador enviado no tiene un formato UUID válido.

401 Unauthorized : Token ausente, inválido, expirado o sesión revocada.

403 Forbidden : El usuario no tiene permisos para consultar la flota.

404 Not Found : No existe un vehículo con el identificador enviado.

500 Internal Server Error : Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya enviado el token.

Validar que corresponda a una sesión existente, no revocada y no expirada.

Comprobar que el rol del usuario tenga permiso para consultar vehículos.

Validar que el id tenga formato UUID.

Buscar el vehículo en la tabla vehicle.

Obtener el tipo y el estado mediante sus relaciones con vehicle_type y vehicle_status.

Responder 404 si el vehículo no existe.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar la información de la flota.

RN 02 Integridad de datos: la información consultada debe corresponder a un vehículo registrado y sus relaciones con los catálogos deben ser válidas.

RN 03 Información del vehículo: El detalle debe permitir identificar la placa, el tipo, la capacidad y las características principales del vehículo.

RN 04 Estado del vehículo: El estado actual debe estar disponible para su consulta.

RN 05 Identificación de vehículos inhabilitados: Un vehículo inhabilitado debe identificarse claramente como no disponible, de acuerdo con HU3.

10. Relación con la base de datos

Este endpoint involucra principalmente tres tablas:

vehicle: Contiene la información del vehículo, como la placa, las capacidades, el estado de actividad y las referencias a los catálogos de tipo y estado.

vehicle_type: Permite consultar el tipo de vehículo, incluyendo su identificador (id), código (code) y nombre (name).

vehicle_status: Permite consultar el estado actual del vehículo, incluyendo su identificador (id), código (code) y nombre (name)

La relación principal es:

vehicle.vehicle_type_id → vehicle_type.id

vehicle.vehicle_status_id → vehicle_status.id

11.Relación con la historia de usuario

HU2 Gestionar vehículos.

HU3 Gestionar estado de vehículos.

Endpoint 8 — Actualizar información del vehículo

metodo y ruta : PATCH /api/v1/vehicles/{id}

autenticacion: Bearer Token

body: si requiere

Para qué sirve? Permite modificar los datos descriptivos de un vehículo ya registrado: placa, tipo y capacidades.

No permite cambiar el estado operativo del vehículo. Para eso se utiliza el Endpoint 10.

Autenticación

Requiere una sesión activa y un Bearer Token válido.

El sistema debe identificar al usuario autenticado y verificar que su rol tenga permisos para modificar la información de la flota.

La matriz de roles y permisos está pendiente de definición en HU1.

Encabezados : http

Authorization: Bearer <access_token>

Content-Type: application/json

Accept: application/json

Cuerpo de la solicitud : json

campo: id

tipo : string

obligatorio: si

descripcion :Identificador único del vehículo que se desea modificar.

Todos los campos son opcionales, pero debe enviarse al menos uno.


```
{
  "plate": "ABC123",
  "vehicleTypeId": "uuid-tipo",
  "capacityKg": 5000,
  "capacityYd3": 7
}
```


campo: id (parámetro de ruta)

tipo: string (UUID)

obligatorio: si

descripcion: Identificador del vehículo que se desea modificar.

campo: plate

tipo: string (máximo 10 caracteres)

obligatorio: no

descripcion: Placa del vehículo.

campo: vehicleTypeId

tipo: string (UUID)

obligatorio: no

descripcion: Tipo de vehículo, debe existir en el catálogo vehicle_type.

campo: capacityKg

tipo: number

obligatorio: no

descripcion: Capacidad del vehículo en kilogramos

campo: capacityYd3

tipo: number

obligatorio: no

descripcion: Capacidad del vehículo en yardas cúbicas.

Respuesta exitosa : Código HTTP: 200 OK

Devuelve el vehículo actualizado con la misma estructura del Endpoint 8.


```
{
  "id": "uuid-vehiculo",
  "plate": "ABC123",
  "vehicleType": {
    "id": "uuid-tipo",
    "name": "Compactador"
  },
  "vehicleStatus": {
    "id": "uuid-estado",
    "name": "Disponible"
  },
  "capacityKg": 5000,
  "capacityYd3": 7,
  "isActive": true,
  "createdAt": "2026-11-02T10:00:00Z",
  "updatedAt": "2026-11-03T09:00:00Z"
}
```


posibles errores :

400 Bad Request : Cuerpo vacío, campos desconocidos o datos con formato inválido.

401 Unauthorized : Token ausente, inválido, expirado o sesión revocada.

403 Forbidden : El usuario no tiene permisos para modificar la flota.

404 Not Found : No existe un vehículo con el identificador enviado.

409 Conflict : La placa ya está registrada en otro vehículo, si se aprueba esa restricción.

500 Internal Server Error : Ocurrió un error inesperado durante el proceso.

Validaciones

Validar que el usuario tenga una sesión activa y un token vigente.

Comprobar que el rol tenga permisos para modificar la flota.

Validar que el parámetro id tenga formato UUID.

Verificar que el vehículo exista en la tabla vehicle.

Comprobar que se haya enviado al menos un campo en el cuerpo de la solicitud.

Rechazar los campos que no estén definidos para esta operación.

Validar que plate cumpla con el formato y la longitud establecidos.

Verificar la unicidad de la placa, si esta restricción es aprobada.

Comprobar que vehicleTypeId corresponda a un tipo existente y activo en vehicle_type.

Validar que capacityKg y capacityYd3 sean valores numéricos mayores que cero, cuando sean proporcionados.

Registrar en audit_log la operación: usuario autenticado, acción UPDATE, entidad vehicle, id del vehículo, valores anteriores (old_values) y nuevos (new_values)

Actualizar updated_at con la fecha y hora de la modificación.

Reglas de negocio

RN-01 Acceso según rol: solamente los roles autorizados pueden modificar la flota.

RN-02 Integridad de datos: el tipo de vehículo debe corresponder a un registro existente.

RN-03 Restricciones de modificación: No se deben permitir cambios que incumplan las reglas definidas para la flota (HU2).

RN-04 Trazabilidad de modificaciones: La modificación debe quedar asociada al usuario que la realizó.

Relación con la base de datos

Este endpoint involucra principalmente tres tablas:

vehicle: se actualizan plate, vehicle_type_id, capacity_kg, capacity_yd3 y updated_at.

vehicle_type: se verifica que el tipo exista y esté activo.

audit_log: se registra quién realizó la modificación y qué valores cambiaro

Relación principal: vehicle.vehicle_type_id → vehicle_type.id.

El usuario que modifica se identifica por la sesión y se guarda en audit_log.

Relación con la historia de usuario

HU2 Gestionar vehículos.

Endpoint 9— Cambiar estado del vehículo

metodo y ruta : PATCH /api/v1/vehicles/{id}/status

autenticacion: Bearer Token

body : Sí requiere.

Para qué sirve? Permite que el jefe de operaciones actualice el estado de un vehículo para identificar si está disponible, en operación o inhabilitado,

por ejemplo por una falla mecánica. Un vehículo inhabilitado queda identificado como no disponible y no puede usarse en una nueva asignación.

Autenticación

Requiere una sesión activa y un Bearer Token válido.

El sistema debe identificar al usuario autenticado y verificar que su rol tenga permisos para modificar el estado operativo de los vehículos.

La matriz de roles y permisos debe corresponder a lo establecido en HU1.

Encabezados : http

Authorization: Bearer <access_token>

Content-Type: application/json

Accept: application/json

Cuerpo de la solicitud : json

```
{
  "vehicleStatusId": "uuid-estado",
```

"reason": "Falla en el sistema hidráulico"

}

campo: id (parámetro de ruta)

tipo: string (UUID)

obligatorio: si

descripcion: Identificador del vehículo al que se le cambia el estado.

campo: vehicleStatusId

tipo: string (UUID)

obligatorio: si

descripcion: Nuevo estado del vehículo, debe existir en el catálogo vehicle_status.

campo: reason

tipo: string

obligatorio:puede quedar como opcional u obligatorio al inhabilitar, segun definamos

descripcion: Se registra en la auditoría cuando se proporciona y debe exigirse cuando el estado nuevo sea inhabilitado, si se aprueba esta regla. Se guarda en audit_log.new_values.

Respuesta exitosa : Código HTTP: 200 OK

{

"id": "uuid-vehiculo",

"plate": "ABC123",

"vehicleStatus": {

"id": "uuid-estado",

"code": "DISABLED",

"name": "Inhabilitado"

},

"updatedAt": "2026-11-03T08:15:00Z"

}

Respuestas de error :

400 Bad Request : Falta vehicleStatusId, tiene formato inválido o falta reason cuando es obligatorio.

401 Unauthorized : Token ausente, inválido, expirado o sesión revocada.

403 Forbidden : El usuario no tiene permisos para cambiar el estado de vehículos.

404 Not Found : No existe el vehículo o el estado enviado.

409 Conflict : La transición de estado no está permitida, por ejemplo inhabilitar un vehículo con una asignación en curso (pendiente de definir).

500 Internal Server Error : Ocurrió un error inesperado durante el proceso.

Validaciones

Antes de actualizar el estado, el backend debe:

Validar que el usuario tenga una sesión activa y un Bearer Token vigente.

Comprobar que el rol tenga permisos para modificar el estado de los vehículos.

Validar que el parámetro id tenga formato UUID.

Verificar que el vehículo exista y se encuentre activo.

Comprobar que vehicleStatusId tenga formato UUID.

Verificar que el estado exista y se encuentre activo en vehicle_status.

Validar que la transición entre estados esté permitida, de acuerdo con las reglas operativas definidas.

Definir el comportamiento cuando el nuevo estado sea igual al estado actual.

Validar el motivo del cambio cuando sea obligatorio.

Antes de inhabilitar el vehículo, verificar que no tenga una asignación en curso, es decir, un registro en assignment con started_at diferente de NULL y ended_at igual a NULL.

Actualizar vehicle_status_id y updated_at en la tabla vehicle.

Registrar la operación en audit_log, incluyendo el usuario responsable, la acción UPDATE, la entidad vehicle, el identificador del vehículo, el estado anterior, el nuevo estado y el motivo cuando se proporcione.

Reglas de negocio

RN 01 Acceso según rol: solamente los roles autorizados pueden cambiar el estado de un vehículo.

RN 02 Disponibilidad operativa: un vehículo inhabilitado debe quedar identificado como no disponible (HU3).

RN 03 Restricción de asignación: un vehículo inhabilitado no debe poder utilizarse en una nueva asignación (HU3 y HU10).

RN 04 Integridad operativa: antes de inhabilitar un vehículo, se debe verificar que no tenga una asignación en curso.

RN 05 Auditoría: el cambio de estado debe quedar asociado al usuario que lo realizó

El sistema debe mostrar el estado actual del vehículo en el listado de flota.

La validación de disponibilidad también debe aplicarse al crear una asignación.

Relación con la base de datos

Este endpoint involucra principalmente tres tablas:

vehicle: se actualizan los campos vehicle_status_id y updated_at.

vehicle_status: permite verificar que el estado solicitado exista y se encuentre activo. El estado inhabilitado debe identificarse mediante el campo code, por ejemplo, DISABLED, y no por el nombre.

assignment: permite verificar si el vehículo tiene una asignación en curso, mediante los campos started_at y ended_at.

audit_log: permite registrar la operación realizada, incluyendo el usuario responsable, la acción, la entidad, los valores anteriores y nuevos, y el motivo del cambio dentro de los campos JSONB correspondientes.

relaciones principales :

vehicle.vehicle_status_id → vehicle_status.id

assignment.vehicle_id → vehicle.id

assignment.assignment_status_id → assignment_status.id

Relación con la historia de usuario

HU3 Gestionar estado de vehículos.

HU10 Asignar recursos a una microruta.

Endpoint 10 — Consultar tipos de vehículo

metodo y ruta : GET /api/v1/vehicle-types

autenticacion: Bearer Token

body : no requiere

Para qué sirve?

Permite consultar los tipos de vehículos registrados en el catálogo de la flota.

Esta información se utiliza para llenar el selector de tipos de vehículo en el formulario de registro y edición, así como para facilitar el filtrado del listado de vehículos.

Por defecto, el sistema debe devolver únicamente los tipos de vehículos activos.

Autenticación Requiere una sesión activa y un Bearer Token válido. El rol del usuario debe tener permiso para acceder al módulo de flota.

Encabezados : http

Authorization: Bearer <access_token>

Accept: application/json

Parámetros de consulta

No requiere cuerpo de solicitud.

Los criterios de filtrado se envían mediante parámetros en la URL

campo: isActive

tipo: boolean

obligatorio: no

descripcion: Filtra por tipos activos o inactivos. Por defecto devuelve solo los activos.

Respuesta exitosa : Código HTTP: 200 OK


```
[
 {
   "id": "uuid-tipo-1",
   "code": "COMPACTOR",
   "name": "Compactador",
   "isActive": true
 },
 {
   "id": "uuid-tipo-2",
   "code": "DUMP_TRUCK",
   "name": "Volqueta",
   "isActive": true
 }
]
```


Respuestas de error :

400 Bad Request: El parámetro isActive tiene un valor o formato inválido

401 Unauthorized : Token ausente, inválido, expirado o sesión revocada.

403 Forbidden : El usuario no tiene permisos para acceder al módulo de flota.

500 Internal Server Error : Ocurrió un error inesperado durante la consulta.

Validaciones

Verificar que el usuario tenga una sesión activa y un Bearer Token válido.

Comprobar que el rol del usuario tenga permisos para consultar los catálogos de la flota.

Validar que el parámetro isActive, cuando sea proporcionado, tenga un valor booleano válido.

Aplicar el filtro correspondiente según el valor recibido.

Devolver únicamente los tipos activos cuando no se proporcione el parámetro.

Consultar los registros correspondientes en la tabla vehicle_type.

Devolver los campos id, code, name e isActive de cada tipo de vehículo.

Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar los catálogos del módulo de flota.

RN 02 Consulta de registros activos: por defecto, el sistema debe devolver únicamente los tipos de vehículos activos.

Los tipos de vehículos deben corresponder a registros existentes en el catálogo.

Los tipos inactivos no deben estar disponibles para nuevas selecciones en los formularios de registro y edición.

Relación con la base de datos

Este endpoint utiliza la siguiente tabla:

vehicle_type : Contiene el catálogo de tipos de vehículos.

La consulta se realiza directamente sobre esta tabla, aplicando el filtro correspondiente al campo is_active.

Relación con la historia de usuario

HU2 Gestionar vehículos.

Este endpoint permite consultar los tipos de vehículos disponibles para:

Llenar el selector de tipos en el formulario de registro.

Llenar el selector de tipos en el formulario de edición.

Facilitar el filtrado de los vehículos registrados en la flota.

Endpoint 11 — Consultar estados de vehículo

metodo y ruta : GET /api/v1/vehicle-statuses

autenticacion : Bearer Token

body: no requiere

Para qué sirve?

Permite consultar los estados operativos registrados en el catálogo de vehículos.

Esta información se utiliza para llenar el selector de estados en el formulario de cambio de estado y para facilitar el filtrado del listado de la flota.

Por defecto, el sistema debe devolver únicamente los estados activos.

Autenticación

Requiere una sesión activa y un Bearer Token válido.

El rol del usuario debe tener permiso para acceder al módulo de flota.

Encabezados : http

Authorization: Bearer <access_token>

Accept: application/json

Parámetros de consulta

No requiere cuerpo de solicitud.

campo:

isActive

tipo: boolean

obligatorio: no

descripcion: Filtra por estados activos o inactivos. Por defecto devuelve solo los activos.

Respuesta exitosa : Código HTTP: 200 OK


```
{
  "data": [
    {
      "id": "uuid-estado",
```

```
    "code": "AVAILABLE",
    "name": "Disponible",
    "isActive": true
   },
   {
    "id": "uuid-estado-2",
    "code": "IN_OPERATION",
    "name": "En operación",
    "isActive": true
   },
   {
    "id": "uuid-estado-3",
    "code": "DISABLED",
    "name": "Inhabilitado",
    "isActive": true
   }
  ]
}
```


Respuestas de error :

401 Unauthorized : Token ausente, inválido, expirado o sesión revocada.

403 Forbidden : El usuario no tiene permisos para acceder al módulo de flota.

500 Internal Server Error : Ocurrió un error inesperado durante la consulta.

Validaciones

Antes de devolver la información, el backend debe:

Verificar que el usuario tenga una sesión activa y un Bearer Token válido.

Comprobar que el rol tenga permisos para consultar los catálogos de la flota.

Validar que el parámetro isActive, cuando sea proporcionado, tenga un valor booleano válido.

Aplicar el filtro correspondiente según el valor recibido.

Devolver únicamente los estados activos cuando no se proporcione el parámetro.

Consultar los registros correspondientes en la tabla vehicle_status.

Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar los catálogos del módulo de flota.

RN 02 Consulta de estados activos: por defecto, el sistema debe devolver únicamente los estados operativos activos.

Los estados deben permitir identificar si un vehículo se encuentra:

Disponible.

En operación.

Inhabilitado.

Un estado inactivo no debe estar disponible para nuevas selecciones en el formulario de cambio de estado.

Los estados deben permitir identificar correctamente la disponibilidad operativa de los vehículos, de acuerdo con HU3.

Relación con la base de datos

Este endpoint utiliza la siguiente tabla:

vehicle_status : Contiene el catálogo de estados del vehículo.

Relación con la historia de usuario

Este endpoint utiliza principalmente la siguiente tabla:

vehicle_status: contiene el catálogo de estados operativos de los vehículos, incluyendo su identificador, nombre y estado de actividad.

La consulta se realiza directamente sobre esta tabla, aplicando el filtro correspondiente al campo is_active.

.

11. Relación con las historias de usuario

HU3 Gestionar estado de vehículos

Este endpoint permite consultar los estados operativos disponibles para:

Llenar el selector de estados en el formulario de cambio de estado.

Identificar los estados disponibles para los vehículos.

Facilitar el filtrado de la flota según el estado operativo

endpoint de personal /recolectores

Endpoint 12 — Listar personal

Método y ruta : GET /api/v1/personnel

body: mo requiere

1. ¿Para qué sirve?

Permite consultar los recolectores registrados en las cuadrillas del sistema.

El listado permite identificar a qué cuadrilla pertenece cada recolector, consultar su nombre completo y conocer si se encuentra activo o inactivo.

También permite filtrar los registros por cuadrilla, buscar por nombre completo y filtrar según el estado de actividad.

Este endpoint está destinado exclusivamente a los recolectores registrados en crew_member. Los conductores se consultan mediante los endpoints específicos del módulo de conductores.

La disponibilidad del personal según el turno no se determina mediante este listado, ya que corresponde a las reglas de disponibilidad de la HU6.

2. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El sistema debe identificar al usuario autenticado y verificar que su rol tenga permisos para consultar la información de los recolectores, de acuerdo con la matriz de roles y permisos definida en HU1.

3**. Encabezados**: HTTP

Authorization: Bearer <token>

Accept: application/json

4. Parámetros de consulta

No requiere un cuerpo de solicitud

Los criterios de búsqueda, filtrado y paginación se envían mediante parámetros en la URL.

campo:crewId

tipo: string uuid

obligatorio : no

descripcion:Filtra los recolectores pertenecientes a una cuadrilla específica.

campo:search

tipo: string

obligatorio: no

descripcion: Busca recolectores por su nombre completo almacenado en full_name.

campo: isActive

tipo:boolean

obligatorio:no

descripcion :Filtra los recolectores según su estado de actividad. Por defecto, el sistema devuelve únicamente los registros activos.

campo:page

tipo: integer

obligatorio:no

descripcion:Número de página. Por defecto es 1.

campo:limit

tipo:integer

obligatorio: no

descripcion:Cantidad de registros por página. Por defecto es 20, con un máximo de 100.

5. Respuesta exitosa

Código HTTP: 200 OK

Devuelve el listado de integrantes que cumplen con los criterios de búsqueda y filtrado.


```
{
  "data": [
    {
      "id": "uuid-integrante",
      "crewId": "uuid-cuadrilla",
      "fullName": "Juan Pérez",
      "isActive": true,
      "createdAt": "2026-11-03T08:00:00Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1
  }
}
```


6. Respuestas de error:

400 Bad Request: Algún parámetro de consulta tiene un formato inválido o se encuentra fuera de los rangos permitidos.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar el personal.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

7**. Validaciones**

Antes de devolver la información, el backend debe:

Verificar que el usuario tenga una sesión activa y un Bearer Token válido.

Comprobar que el rol del usuario tenga permisos para consultar los recolectores

Validar que crewId tenga formato UUID cuando sea proporcionado.

Verificar que la cuadrilla utilizada como filtro exista en la tabla crew

Validar que search tenga un formato permitido cuando se proporcione.

Validar que isActive tenga un valor booleano válido.

Validar que page sea un número entero mayor o igual a 1.

Validar que limit sea un número entero mayor o igual a 1 y no supere 100.

Aplicar los valores predeterminados de paginación cuando no se proporcionen.

Consultar los registros de crew_member y establecer la relación con crew.

Aplicar los filtros de búsqueda, cuadrilla y estado de actividad.

Realizar la paginación de los resultados.

Devolver los registros correspondientes junto con los metadatos de paginación

8. Reglas de negocio

RN 01 Acceso según rol: Solamente los usuarios autorizados pueden consultar la información de los recolectores.

RN 02 Asociación a cuadrillas: Cada recolector debe estar asociado a una cuadrilla existente.

RN 03 Consulta de personal activo: Por defecto, el sistema debe devolver únicamente los recolectores activos.

RN 04 Búsqueda por nombre: La búsqueda por texto no debe distinguir entre mayúsculas y minúsculas.

RN 05 Separación de tipos de personal: Este endpoint consulta exclusivamente los registros de crew_member. Los conductores se gestionan mediante sus endpoints específicos.

RN 06 Estado de actividad: El campo isActive indica si el registro del recolector está activo, pero no representa su disponibilidad para un turno específico

9. Relación con la base de datos

Este endpoint involucra principalmente dos tablas:

crew_member: Contiene los recolectores registrados, incluyendo su identificador, cuadrilla, nombre completo, estado de actividad y fechas de creación y actualización

crew: Permite identificar la cuadrilla a la que pertenece cada recolector.

Las relaciones principales son:

crew_member.crew_id →crew.id

10. Relación con las historias de usuario

HU4 Gestionar personal operativo

Este endpoint permite cumplir los siguientes criterios:

Consultar el personal operativo registrado.

Consultar los recolectores registrados.

Identificar el nombre completo de cada recolector.

Identificar la cuadrilla a la que pertenece.

Filtrar los recolectores por cuadrilla y estado de actividad

Endpoint 13 — Consultar integrante del personal

metodo y ruta : GET /api/v1/personnel/{id}

body: no requiere

autenticacion: Bearer Token

2. ¿Para qué sirve?

Permite consultar el detalle de un recolector específico registrado en el sistema.

Devuelve su identificador, nombre completo, cuadrilla a la que pertenece y estado de actividad.

Se utiliza para mostrar la información individual del recolector y consultar sus datos antes de realizar una modificación.

Este endpoint consulta exclusivamente los registros almacenados en crew_member. Los conductores se consultan mediante el endpoint correspondiente al módulo de conductores.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El sistema debe identificar al usuario autenticado y verificar que su rol tenga permisos para consultar la información de los recolectores

encabezado: http

Authorization: Bearer <token>

Accept: application/json

5. Parámetros de ruta

No requiere un cuerpo de solicitud

El identificador del integrante se envía directamente en la URL.

campo:id

tipo:string uuid

obligatorio:si

descripcion:Identificador único recolector que se desea consultar.

6. Respuesta exitosa

Código HTTP: 200 OK

Devuelve la información detallada del integrante consultado

{

```
  "id": "uuid-integrante",
  "crewId": "uuid-cuadrilla",
  "crewName": "Cuadrilla Norte",
  "fullName": "Juan Pérez",
  "isActive": true,
  "createdAt": "2026-11-03T08:00:00Z",
  "updatedAt": "2026-11-03T08:00:00Z"
}
```


Respuestas de error:

400 Bad Request: El identificador enviado no tiene un formato UUID válido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar el personal.

404 Not Found: No existe un integrante con el identificador enviado.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya proporcionado un token de autenticación.

Validar que corresponda a una sesión existente, vigente y no revocada.

Comprobar que el rol del usuario tenga permisos para consultar el personal.

Validar que el parámetro id tenga un formato UUID válido.

Buscar el integrante en la tabla crew_member.

Obtener la información de la cuadrilla mediante su relación con crew.

Responder con código 404 Not Found si el integrante no existe.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar la información del personal operativo.

RN 02 Identificación del integrante: el detalle debe permitir identificar al integrante mediante su nombre completo y la cuadrilla a la que pertenece.

RN 01 Acceso según rol: Solamente los usuarios autorizados pueden consultar la información de los recolectores.

RN 02 Identificación del integrante: El detalle debe permitir identificar al recolector mediante su nombre completo y la cuadrilla a la que pertenece.

RN 03 Consulta de registros inactivos: Un recolector inactivo puede consultarse, pero el sistema debe indicar claramente su estado mediante el campo isActive.

RN 04 Separación de tipos de personal: La consulta corresponde exclusivamente a los recolectores registrados en crew_member.

La consulta del detalle no debe confundirse con la disponibilidad del recolector para un turno específico.

10. Relación con la base de datos

Este endpoint involucra principalmente tres tablas:

crew_member: contiene la información del integrante, incluyendo su nombre completo, estado de actividad y fecha de creación.

crew: permite consultar la cuadrilla a la que pertenece el integrante.

11. Relación con las historias de usuario

HU4 Gestionar personal operativo

Este endpoint permite consultar la información detallada de un conductor o recolector registrado, incluyendo su identificación y tipo de personal.

Consultar la información individual de un recolector registrado.

Identificar su nombre completo.

Consultar la cuadrilla a la que pertenece.

Identificar si se encuentra activo o inactivo.

Endpoint 14 — Registrar integrante del personal

Método: POST Ruta: /api/v1/personnel

Autenticación: Bearer Token

Body: Sí requiere.

1. ¿Para qué sirve?

Permite que el jefe de operaciones registre un nuevo recolector y lo asocie a una cuadrilla existente.

El registro almacena el nombre completo del recolector y establece su relación con la cuadrilla correspondiente.

Los recolectores no requieren una cuenta de usuario para ser registrados, ya que su información operativa se almacena directamente en la tabla crew_member.

Esta información será utilizada posteriormente para consultar los integrantes de las cuadrillas y organizar los recursos operativos.

2. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El sistema debe identificar al usuario autenticado y verificar que su rol tenga permisos para registrar recolectores

La matriz de roles y permisos está pendiente de definición en HU1.

encabezado: http

Authorization: Bearer <token>

Content-Type: application/json

Accept: application/json

3. Cuerpo de la solicitud

El registro requiere un cuerpo de solicitud en formato JSON.

```
{
  "crewId": "uuid-cuadrilla",
  "fullName": "Juan Pérez"
```

}

campo:crewId

tipo:string (UUID)

obligatorio:si

descripcion:Identificador de la cuadrilla existente a la que pertenecerá el recolector

campo:fullName

tipo:string

obligatorio:si

descripcion:Nombre completo del recolector que se desea registrar.

4. Respuesta exitosa

Código HTTP: 201 Created

Devuelve el integrante creado con su información correspondiente

{
  "id": "uuid-integrante",
  "crewId": "uuid-cuadrilla",
  "crewName": "Cuadrilla Norte",
  "fullName": "Juan Pérez",
  "isActive": true,
  "createdAt": "2026-11-04T08:00:00Z",
  "updatedAt": "2026-11-04T08:00:00Z"
}

5. Respuestas de error:

400 Bad Request: Faltan campos obligatorios o los datos enviados no cumplen con el formato esperado.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para registrar personal.

404 Not Found: El tipo de personal o el tipo de documento enviado no existe.

409 Conflict: Ya existe un integrante con el mismo tipo y número de documento, según la restricción de unicidad que se apruebe.

500 Internal Server Error: Ocurrió un error inesperado durante el registro.

6. Validaciones

Antes de registrar el integrante, el backend debe:

Verificar que se haya proporcionado un token de autenticación.

Validar que corresponda a una sesión existente, vigente y no revocada.

Comprobar que el rol del usuario tenga permisos para registrar recolectores.

Verificar que se hayan enviado todos los campos obligatorios.

Validar que crewId tenga formato UUID.

Verificar que la cuadrilla correspondiente exista en la tabla crew.

Validar que fullName no esté vacío.

Verificar que el nombre completo no supere las restricciones de almacenamiento de la columna full_name.

Crear el registro en crew_member, asociándolo con la cuadrilla correspondiente.

Establecer is_active = true mediante el valor predeterminado de la columna.

Registrar las fechas de creación y actualización mediante los valores predeterminados definidos en la tabla.

7. Reglas de negocio

RN 01 Acceso según rol: Solamente los roles autorizados pueden registrar recolectores.

RN 02 Asociación a cuadrillas: Todo recolector debe pertenecer a una cuadrilla existente, ya que crew_id es obligatorio en crew_member.

RN 03 Integridad de datos: El nombre completo del recolector es obligatorio.

RN 04 Estado inicial: Los recolectores nuevos deben crearse inicialmente como activos, de acuerdo con el valor predeterminado de is_active.

RN 05 Sin cuenta de usuario: El registro de un recolector no debe crear un registro en user_account, user_data ni profile, ya que los recolectores no tienen acceso individual al sistema según el modelo definido.

9. Relación con la base de datos

Este endpoint involucra principalmente cuatro tablas:

crew_member: almacena el nuevo registro del integrante , incluyendo su identificador, cuadrilla, nombre completo, estado de actividad y fecha de creación.

crew: permite verificar que la cuadrilla seleccionada exista y se encuentre registrada en el sistema.

10. Relación con las historias de usuario

hu4-gestionar personal operativo

Este endpoint permite cumplir los siguientes criterios:

Registrar nuevos recolectores.

Asociar recolectores a una cuadrilla existente.

Almacenar el nombre completo de cada recolector.

Establecer el estado inicial de actividad del recolector.

HU5 Gestionar cuadrillas y turnos

Permite incorporar recolectores a una cuadrilla existente para conformar el personal operativo.

Endpoint 15 — Actualizar información del integrante del personal

Método: PATCH

Ruta: /api/v1/personnel/{id}

Módulo: Personal

Autenticación: Bearer Token

Body: Sí requiere.

Operación: Modificar la información de un integrante registrado y permitir su activación o desactivación lógica.

1. ¿Para qué sirve?

Permite actualizar la información de un recolector previamente registrado en el sistema.

La modificación puede incluir el nombre completo, la cuadrilla a la que pertenece y su estado de actividad.

También permite desactivar o reactivar al recolector mediante el campo isActive.

La desactivación es lógica, es decir, el registro no se elimina físicamente de la base de datos. De esta manera se conserva la información histórica del personal y sus relaciones con las cuadrillas y las operaciones anteriores.

Un recolector inactivo no debe ser considerado para nuevas operaciones, de acuerdo con las reglas de disponibilidad y conformación de cuadrillas.

Este endpoint modifica exclusivamente registros de crew_member. Los datos de los conductores se actualizan mediante los endpoints específicos del módulo de conductores.

2. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El sistema debe identificar al usuario autenticado y verificar que su rol tenga permisos para modificar la información del personal operativo.

La matriz de roles y permisos debe corresponder a lo establecido en HU1.

encabezado: http

Authorization: Bearer <token>

Content-Type: application/json

Accept: application/json

3. Parámetros de ruta y cuerpo de la solicitud

campo:id

tipo:string uuid

obligatorio:si

descripcion:Identificador único del integrante que se desea modificar.

4. Cuerpo de la solicitud :

Todos los campos son opcionales individualmente, pero debe enviarse al menos uno.

{

"crewId": "uuid-nueva-cuadrilla",

"fullName": "Juan Carlos Pérez",

"isActive": true

}

5. Respuesta exitosa

Código HTTP: 200 OK

Devuelve la información del integrante actualizado con la misma estructura de respuesta del Endpoint 14 — Consultar integrante del personal.

{

"id": "uuid-integrante",

"crewId": "uuid-nueva-cuadrilla",

"crewName": "Cuadrilla Sur",

"fullName": "Juan Carlos Pérez",

"isActive": true,

"createdAt": "2026-11-03T08:00:00Z",

"updatedAt": "2026-11-04T09:00:00Z"

}

6. Respuestas de error:

400 Bad Request: El cuerpo está vacío, contiene campos desconocidos o presenta datos con formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para modificar el personal.

404 Not Found: No existe el integrante o alguno de los catálogos referenciados.

409 Conflict: El tipo y número de documento ya pertenecen a otro integrante, o el integrante no puede desactivarse porque tiene una asignación vigente, si esta restricción es aprobada.

500 Internal Server Error: Ocurrió un error inesperado durante el proceso.

7. Validaciones

Antes de actualizar la información, el backend debe:

Verificar que se haya proporcionado un token de autenticación.

Validar que corresponda a una sesión existente, vigente y no revocada.

Comprobar que el rol del usuario tenga permisos para modificar los recolectores.

Validar que el parámetro id tenga formato UUID.

Verificar que el recolector exista en la tabla crew_member.

Comprobar que se haya enviado al menos un campo en el cuerpo de la solicitud.

Rechazar los campos que no estén definidos para esta operación.

Validar que crewId, cuando se envíe, tenga formato UUID.

Verificar que la nueva cuadrilla exista en la tabla crew.

Validar que fullName, cuando se envíe, no esté vacío y cumpla las restricciones de almacenamiento de full_name.

Validar que isActive tenga un valor booleano válido cuando se proporcione.

Verificar las restricciones operativas definidas para cambiar de cuadrilla o desactivar al recolector.

Actualizar únicamente los campos enviados.

Actualizar updated_at con la fecha y hora de la modificación.

Registrar los valores anteriores de la modificación en crew_member_log, de acuerdo con el mecanismo de auditoría implementado.

8. Reglas de negocio

RN 01 Acceso según rol: Solamente los roles autorizados pueden modificar la información de los recolectores.

RN 02 Integridad de datos: La cuadrilla referenciada debe corresponder a un registro existente en crew.

RN 03 Desactivación lógica: La desactivación del recolector debe realizarse mediante el campo is_active, sin eliminar físicamente el registro.

RN 04 Conservación histórica: La información histórica del recolector debe conservarse para no afectar los registros de cuadrillas y operaciones anteriores.

RN 05 Estado de actividad: Un recolector inactivo no debe incluirse en nuevas operaciones que requieran personal activo.

RN 06 Actualización parcial: El sistema solamente debe modificar los campos enviados en la solicitud, conservando los valores de los campos que no se proporcionen.

9. Relación con la base de datos

Este endpoint involucra principalmente cuatro tablas:

crew_member: Permite actualizar la información del recolector, incluyendo crew_id, full_name, is_active y updated_at.

crew: Permite verificar que la nueva cuadrilla referenciada exista.

10. Relación con las historias de usuario

HU4 Gestionar personal operativo

Este endpoint permite:

Modificar el nombre completo de los recolectores registrados.

Actualizar la cuadrilla a la que pertenecen.

Activar o desactivar recolectores sin eliminarlos físicamente.

HU5 Gestionar cuadrillas y turnos

Este endpoint permite actualizar la asociación de un recolector con una cuadrilla existente, de acuerdo con las reglas de conformación de cuadrillas.

HU6 Gestionar disponibilidad del personal

El estado de actividad del recolector debe considerarse al determinar si puede participar en una operación. Sin embargo, isActive no representa por sí solo la disponibilidad para un turno específico.

CONDUCTORES

Endpoint 16. Registrar un conductor

Método y ruta: POST /api/v1/drivers

1. ¿Para qué sirve?

Permite registrar un nuevo conductor en el sistema, creando su cuenta de usuario, sus datos personales, su perfil con el rol DRIVER y su relación laboral con una cuadrilla existente.

A diferencia de los recolectores, los conductores sí tienen acceso al sistema mediante una cuenta de usuario, por lo que deben contar con credenciales de autenticación.

El registro debe realizarse de manera conjunta para garantizar que la cuenta, los datos personales, el perfil y la vinculación laboral queden correctamente relacionados

parametro de entrada

{

"documentType": "CC",

"documentId": "1234567890",

"firstName": "Carlos",

"lastName": "Pérez",

"email": "carlos@example.com",

"phone": "3001234567",

"crewId": 1

}

campo:documentype

descripción:ipo de documento de identidad del conductor. Los valores permitidos corresponden al ENUM document_type_enum: CC, CE y PASSPORT

campo:docunetid

descripción:número de documento de identidad del conductor. Se almacena en user_account.document_id.

campo:firstname

descripción:nombres del conductor. Se almacena en user_data

campo:lastName

descripción:apellidos del conductor. Se almacena en user_data

campo:email

descripción:correo electronico del conductor

campo:phone

descripción:Nuemero de contacto

campo:password

descripcion:contraseña inicial de la cuenta del conductor. El backend debe generar su hash y almacenarlo en user_account.pwd_hash. Nunca debe guardarse en texto plano.

campo:crewId

descripción:iidentificador UUID de la cuadrilla a la que se vinculará el conductor. Se relaciona con driver.crew_id.

Respuesta exitosa

codigo http: 201


```
{
  "status": "success",
  "message": "Conductor registrado correctamente",
  "data": {
    "id": 1,
    "documentType": "CC",
    "documentId": "1234567890",
    "firstName": "Carlos",
    "lastName": "Pérez",
```

```
  "email": "carlos@example.com",
  "phone": "3001234567",
  "role": "DRIVER",
  "crew": {
    "id": 1,
    "name": "Cuadrilla Norte"
  },
  "startedAt": "2026-11-05T13:00:00",
  "isActive": true
 }
}
```


6. respuestas de error:

400 Bad Request: El cuerpo está vacío, contiene campos desconocidos o presenta datos con formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para modificar el personal.

404 Not Found: No existe el integrante o alguno de los catálogos referenciados.

409 Conflict: El tipo y número de documento ya pertenecen a otro integrante, o el integrante no puede desactivarse porque tiene una asignación vigente, si esta restricción es aprobada.

500 Internal Server Error: Ocurrió un error inesperado durante el proceso.

7. Validaciones

Antes de registrar la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada mediante la tabla session.

Comprobar que el rol del usuario tenga permisos para registrar conductores.

Validar que los campos obligatorios estén completos.

Validar que documentType corresponda a un valor permitido por document_type_enum.

Verificar que el correo electrónico tenga un formato válido.

Comprobar que la cuadrilla indicada mediante crewId exista.

Verificar que el documento y el correo no estén registrados previamente, conforme a las restricciones de unicidad que apruebe el equipo.

Validar que la contraseña cumpla con las políticas de seguridad establecidas.

Ejecutar el registro dentro de una transacción para evitar registros incompletos si alguna operación falla.

8. Reglas de negocio

RN 01. Acceso según rol: solamente los usuarios autorizados pueden registrar conductores.

RN 02. Registro de conductores: cada conductor debe contar con una cuenta de usuario, sus datos personales y un perfil con el rol DRIVER.

RN 03. Vinculación con cuadrilla: todo conductor registrado mediante este endpoint debe quedar asociado a una cuadrilla existente.

RN 04. Estado inicial de la cuenta: la cuenta debe crearse con el estado ACTIVE, de acuerdo con el valor predeterminado de user_status_enum.

RN 05. Estado inicial de la vinculación: la relación laboral debe crearse activa (is_active = true).

RN 06. Fecha de vinculación: el campo started_at registra la fecha de inicio de la relación laboral y utiliza por defecto la fecha y hora actual.

RN 07. Seguridad de credenciales: las contraseñas deben almacenarse mediante un mecanismo de hash en pwd_hash, nunca en texto plano.

RN 08. Integridad del registro: la creación de la cuenta, los datos personales, el perfil y la vinculación laboral deben realizarse dentro de una misma transacción.

9. Relación con la base de datos

Este endpoint involucra principalmente cinco tablas:

user_account: almacena la información de autenticación del conductor.

user_data: contiene sus datos personales y de contacto.

profile: registra el rol asignado a la cuenta.

driver: establece la relación laboral entre el conductor y la cuadrilla.

crew: permite verificar la existencia de la cuadrilla seleccionada.

Las relaciones principales son:

user_data.id → user_account.id

profile.user_account_id → user_account.id

driver.user_data_id → user_data.id

driver.crew_id → crew.id

10. Relación con las historias de usuario

HU4 Gestionar personal operativo

Este endpoint cumple con el criterio de permitir registrar conductores como integrantes del personal operativo, asociándolos a una cuenta de usuario, un perfil con el rol DRIVER y una cuadrilla.

También contribuye a identificar el tipo de personal registrado y consultar posteriormente su información dentro del sistema.

HU1 Acceso según rol

Se relaciona con esta historia porque el conductor debe contar con una cuenta de usuario y un perfil que permita identificar sus permisos de acceso mediante el rol DRIVER.

Endpoint 17/ listar conductores

metodo y rura :GET /api/v1/drivers

1. ¿Para qué sirve?

Permite consultar el listado de conductores registrados en el sistema, incluyendo sus datos personales, la cuadrilla a la que pertenecen y el estado de su vinculación laboral.

2. Parámetros de entrada

Este endpoint no requiere un cuerpo de solicitud. Puede recibir los siguientes parámetros opcionales mediante la URL:

parámetro:crewId

descripcion:filtra los conductores pertenecientes a una cuadrilla especifica

parametro:isActive

descripcion:fiiltra por estado de la vinculación laboral

parametro:search

descripcion:permite buscar por nombre,apellido,documento o correo electronico

parametro:page

descripcion:numero de pagina que se desea consultar

parametro:limit

descripcion:cantidad de registros por pagina

ejemplo de consulta:

GET /api/v1/drivers?crewId=1&isActive=true&page=1&limit=10

4.Respuesta exitosa

codigo http: 200 ok


```
{
  "status": "success",
  "message": "Conductores consultados correctamente",
  "data": [
    {
      "id": 1,
      "documentType": "CC",
      "documentId": "1234567890",
      "firstName": "Carlos",
      "lastName": "Pérez",
      "email": "carlos@example.com",
      "phone": "3001234567",
      "crew": {
```

```
      "id": 1,
      "name": "Cuadrilla Norte"
    },
    "startedAt": "2026-11-05T13:00:00",
    "endedAt": null,
    "isActive": true
  }
 ],
 "pagination": {
  "page": 1,
  "limit": 10,
  "total": 1,
  "totalPages": 1
 }
}
```


6.errores

8. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada.

Comprobar que el rol del usuario tenga permiso para consultar conductores.

Validar los parámetros de consulta y la paginación.

Verificar que crewId, si se proporciona, corresponda a una cuadrilla existente.

Validar que isActive tenga un valor booleano válido.

Consultar la tabla driver.

Obtener los datos personales desde user_data.

Consultar la información de la cuenta en user_account.

Obtener la información de la cuadrilla asociada mediante crew.

Aplicar los filtros y la paginación solicitados.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar conductores.

RN 02 Consulta de conductores: se deben poder consultar los conductores registrados en el sistema, según HU4.

RN 03 Separación del personal: este endpoint consulta únicamente los conductores registrados mediante driver. Los recolectores se gestionan por separado mediante crew_member.

RN 04 Estado de vinculación: el filtro isActive permite consultar conductores según el estado de su relación laboral con la cuadrilla.

RN 05 Filtro por cuadrilla: se debe permitir consultar los conductores asociados a una cuadrilla específica.

RN 06 Resultados paginados: el listado debe permitir controlar la cantidad de registros devueltos mediante los parámetros de paginación.

10. Relación con la base de datos

Este endpoint involucra principalmente cinco tablas:

driver: contiene los registros de vinculación laboral de los conductores.

user_data: proporciona los datos personales.

user_account: permite consultar la información relacionada con la cuenta.

profile: permite identificar el rol del usuario.

crew: proporciona la información de la cuadrilla asociada.

Las relaciones principales son:

driver.user_data_id → user_data.id

user_data.id → user_account.id

profile.user_account_id → user_account.id

driver.crew_id → crew.id

11. Relación con las historias de usuario

HU4 Gestionar personal operativo

Este endpoint cumple el criterio de permitir consultar y listar los conductores registrados, facilitando la identificación de sus datos personales y de la cuadrilla a la que pertenecen.

Endpoint 18/ consultar el detalle del conductor

Método y ruta: GET /api/v1/drivers/{id}

1. Para qué sirve?

Permite consultar la información detallada de un conductor específico mediante su identificador. Incluye sus datos personales, información de su cuenta, el rol asignado y los datos de la cuadrilla con la que mantiene su relación laboral.

3. parametros de entrada

parametro:id

descripcion:identificador del registro del conductor en la tabla driver

ejemplo de consulta:

GET /api/v1/drivers/1

4.Respuesta exitosa

http:200 ok

{

"status": "success",

"message": "Conductor consultado correctamente",

"data": {

"id": 1,

"documentType": "CC",

"documentId": "1234567890",

"firstName": "Carlos",

```
   "lastName": "Pérez",
   "email": "carlos@example.com",
   "phone": "3001234567",
   "role": "DRIVER",
   "crew": {
    "id": 1,
    "name": "Cuadrilla Norte"
   },
   "startedAt": "2026-11-05T13:00:00",
   "endedAt": null,
   "isActive": true,
   "createdAt": "2026-11-05T13:00:00",
   "updatedAt": "2026-11-05T13:00:00"
 }
}
```


6.respuestas de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada.

Comprobar que el rol del usuario tenga permiso para consultar el detalle de los conductores.

Validar que el identificador recibido en la ruta sea válido.

Consultar el registro correspondiente en la tabla driver.

Obtener los datos personales del conductor desde user_data.

Consultar la información de la cuenta en user_account.

Verificar el perfil asociado en profile.

Obtener la información de la cuadrilla mediante crew_id.

Evitar devolver información sensible, como contraseñas o tokens de autenticación.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar el detalle de los conductores.

RN 02 Consulta individual: se debe permitir consultar la información de un conductor específico mediante su identificador.

RN 03 Información relacionada: la respuesta debe incluir los datos personales del conductor, su información de vinculación laboral y la cuadrilla asociada.

RN 04 Seguridad de la información: no se deben exponer contraseñas, hashes ni tokens almacenados en las tablas de autenticación.

RN 05 Identificación del conductor: el identificador utilizado corresponde al registro del conductor en la tabla driver.

10. Relación con la base de datos

Este endpoint involucra principalmente cinco tablas:

driver: permite identificar el registro del conductor y su vinculación laboral.

user_data: contiene los datos personales y de contacto.

user_account: almacena la información de la cuenta asociada.

profile: permite consultar el rol asignado.

crew: proporciona los datos de la cuadrilla a la que pertenece.

Las relaciones principales son:

driver.user_data_id → user_data.id

user_data.id → user_account.id

profile.user_account_id → user_account.id

driver.crew_id → crew.id

11. Relación con las historias de usuario

HU4 Gestionar personal operativo

Este endpoint cumple el criterio de permitir consultar individualmente la información de un conductor registrado, incluyendo sus datos personales y su relación con una cuadrilla

Endpoint 19/Actualizar la informacion de un conductor

Método y ruta: PATCH /api/v1/drivers/{id}

1. ¿Para qué sirve?

Permite actualizar parcialmente la información personal y los datos de contacto de un conductor registrado en el sistema, sin necesidad de enviar todos sus datos nuevamente.

La actualización contempla los datos almacenados en user_data y, cuando corresponda, el estado de su vinculación laboral en driver.

3. Parámetro de entrada

parámetro: id

descripción: identificador del registro del conductor en la tabla driver.

cuerpo de la solicitud: JSON

{

 "firstName": "Carlos Andrés",

 "lastName": "Pérez Gómez",

 "email": "carlosandres@example.com",

 "phone": "3019876543",

 "isActive": true

}

campo:FirstName

descripcion:Actualiza los nombres del conductor

campo:LastName

descripcion:Actualiza los apellidos del conductor

campo:email

descripcion:Actualiza el correo electrónico

campo:phone

descripcion:Actualiza el numero de contacto

campo:isActive

descripcion:Actualiza el estado de la vinculacion laboral en driver

4.Respuesta exitosa

codigo http:200 ok


```
{
 "status": "success",
 "message": "Información del conductor actualizada correctamente",
 "data": {
  "id": 1,
  "firstName": "Carlos Andrés",
  "lastName": "Pérez Gómez",
  "email": "carlosandres@example.com",
  "phone": "3019876543",
  "crew": {
   "id": 1,
   "name": "Cuadrilla Norte"
  },
  "isActive": true,
  "updatedAt": "2026-11-05T14:00:00"
```

}

}

6.Respuesta de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de actualizar la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada.

Comprobar que el rol del usuario tenga permiso para actualizar conductores.

Validar que el identificador recibido en la ruta sea válido.

Verificar que el conductor exista en la tabla driver.

Validar que los campos enviados cumplan con los formatos y restricciones correspondientes.

Comprobar que el correo electrónico tenga un formato válido, si se proporciona.

Verificar que el correo no esté asociado a otra cuenta, si se establece esta restricción.

Actualizar los datos personales en user_data, cuando corresponda.

Actualizar el estado de la vinculación laboral en driver, si se envía isActive.

Registrar los cambios mediante el mecanismo de auditoría correspondiente, si está implementado

.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden actualizar la información de los conductores.

RN 02 Actualización parcial: se deben actualizar únicamente los campos enviados en la solicitud, conservando los valores de los campos que no se modifiquen.

RN 03 Separación de información: los datos personales y de contacto se actualizan en user_data, mientras que el estado de la vinculación laboral se actualiza en driver.

RN 04 Estado de vinculación: el campo isActive permite activar o desactivar la relación laboral del conductor.

RN 05 Conservación de la cuenta: la actualización de los datos del conductor no debe eliminar ni modificar su cuenta de usuario o su perfil DRIVER.

RN 06 Cambio de cuadrilla: la modificación de la cuadrilla no se contempla en este endpoint hasta que se defina el procedimiento para conservar el historial de las vinculaciones laborales.

RN 07 Seguridad de credenciales: este endpoint no permite modificar contraseñas ni exponer información de autenticación.

10. Relación con la base de datos

Este endpoint involucra principalmente tres tablas:

driver: permite consultar y actualizar el estado de la vinculación laboral.

user_data: almacena los datos personales y de contacto que pueden modificarse.

user_account: permite identificar la cuenta asociada al conductor.

Las relaciones principales son:

driver.user_data_id → user_data.id

user_data.id → user_account.id

Para la trazabilidad de los cambios también se dispone de:

driver_log: registra los cambios relacionados con la vinculación laboral.

user_data_log: registra los cambios relacionados con los datos personales.

Estas tablas contemplan información como la acción realizada, los valores anteriores, el actor y la fecha del registro. Su generación debe estar implementada en el mecanismo de auditoría del sistema.

11. Relación con las historias de usuario

HU4 Gestionar personal operativo

Este endpoint cumple el criterio de permitir actualizar la información de los conductores registrados, incluyendo sus datos personales, de contacto y el estado de su vinculación laboral.

CUADRILLAS:

Endpoint 20/ Crear una cuadrilla

Método y ruta: POST /api/v1/crews

1. ¿Para qué sirve?

Permite registrar una nueva cuadrilla de trabajo en el sistema, asignándole un nombre y estableciendo su estado inicial.

Las cuadrillas permiten organizar al personal operativo que participará en las actividades de recolección.

3. Parámetro de entrada

cuerpo de la solicitud JSON

{

"name": "Cuadrilla Norte"

}

Descripción de los campos:

Campo: name

Descripción: nombre asignado a la cuadrilla. Se almacena en crew.name.

4.respuesta exitosa

Código http: 201

{

"status": "success",

"message": "Cuadrilla creada correctamente",

"data": {

"id": "550e8400-e29b-41d4-a716-446655440000",

"name": "Cuadrilla Norte",

"isActive": true,

"createdAt": "2026-11-05T13:00:00Z"

}

}

5.respuesta de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

6. Validaciones

Antes de registrar la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada mediante la tabla session.

Comprobar que el rol del usuario tenga permisos para crear cuadrillas.

Verificar que el campo name esté completo.

Validar que el nombre no contenga únicamente espacios en blanco.

Comprobar que la información cumpla con las restricciones definidas para el campo name.

7. Reglas de negocio

RN 01. Acceso según rol: solamente los usuarios autorizados pueden crear cuadrillas.

RN 02. Registro de cuadrillas: toda cuadrilla debe contar con un nombre que permita identificarla dentro del sistema.

RN 03. Estado inicial: las cuadrillas deben crearse activas (is_active = true), de acuerdo con el valor predeterminado de la tabla crew.

RN 04. Identificación: cada cuadrilla debe contar con un identificador UUID generado automáticamente por la base de datos.

RN 05. Fecha de creación: el campo created_at registra automáticamente la fecha y hora de creación de la cuadrilla.

8. Relación con la base de datos

Este endpoint utiliza la tabla:

crew: almacena la información de la cuadrilla.

Campos involucrados:

id: identificador UUID de la cuadrilla.

name: nombre de la cuadrilla.

is_active: estado de la cuadrilla.

created_at: fecha de creación.

updated_at: fecha de actualización.

9. Relación con las historias de usuario

HU5 Gestionar cuadrillas y turnos

Este endpoint permite registrar las cuadrillas que posteriormente serán conformadas con personal operativo y asociadas a los turnos correspondientes.

Contribuye al cumplimiento del criterio que establece que el sistema debe permitir conformar cuadrillas con personal registrado.

Endpoint 21/listar cuadrillas

Método y ruta: GET /api/v1/crews

1. ¿Para qué sirve?

Permite consultar el listado de cuadrillas registradas en el sistema, mostrando su información general y estado actual.

La consulta facilita la administración de las cuadrillas y permite identificar cuáles se encuentran activas o inactivas.

2. Parámetros de entrada

Este endpoint permite utilizar parámetros de consulta opcionales para filtrar los resultados.

Ejemplo de solicitud:

GET /api/v1/crews?isActive=true&page=0&size=10

Descripción de los parámetros**:**

Campo: isActive

Descripción: permite filtrar las cuadrillas según su estado. Acepta los valores true o false.

Campo: page

Descripción: indica el número de página que se desea consultar. Su valor predeterminado puede ser 0.

Campo: size

Descripción: establece la cantidad máxima de registros que se devolverán por página.

3. Respuesta exitosa :http 200 ok


```
{
  "status": "success",
  "message": "Cuadrillas consultadas correctamente",
  "data": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "name": "Cuadrilla Norte",
      "isActive": true,
      "createdAt": "2026-11-05T13:00:00Z"
    },
    {
      "id": "550e8400-e29b-41d4-a716-446655440002",
      "name": "Cuadrilla Sur",
      "isActive": false,
      "createdAt": "2026-11-05T14:00:00Z"
    }
  ],
  "pagination": {
    "page": 0,
```

```
  "size": 10,
  "totalElements": 2,
  "totalPages": 1
 }
}
```


4.respuesta de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

5. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada mediante la tabla session.

Comprobar que el rol del usuario tenga permisos para consultar cuadrillas.

Validar que el parámetro isActive, cuando se envíe, contenga un valor booleano válido.

Validar que los parámetros de paginación tengan valores permitidos.

Consultar únicamente los registros que cumplan con los filtros enviados.

6. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar las cuadrillas.

RN 02 Consulta de cuadrillas: el sistema debe permitir visualizar las cuadrillas registradas.

RN 03 Identificación del estado: cada cuadrilla debe mostrar si se encuentra activa o inactiva.

RN 04 Filtrado: el sistema debe permitir filtrar las cuadrillas según su estado.

RN 05 Paginación: cuando la cantidad de registros sea elevada, la consulta debe permitir dividir los resultados en páginas.

7. Relación con la base de datos

Este endpoint utiliza la tabla:

crew: contiene los registros de las cuadrillas.

Campos involucrados:

id

name

is_active

created_at

updated_at

8. Relación con las historias de usuario

HU5 Gestionar cuadrillas y turnos

Este endpoint permite consultar las cuadrillas registradas para facilitar su organización y administración, así como identificar cuáles se encuentran activas para la planificación operativa.

Endpoint 22/consultar cuadrilla por id

Método y ruta: GET /api/v1/crews/{id}

1. ¿Para qué sirve?

Permite consultar la información detallada de una cuadrilla específica mediante su identificador UUID.

La respuesta incluye los datos generales de la cuadrilla y el personal asociado a ella, permitiendo identificar los conductores y recolectores que la conforman.

2. Parámetros de entrada

Parámetro de ruta: id

Descripción: identificador UUID de la cuadrilla que se desea consultar.

Ejemplo de solicitud:

GET /api/v1/crews/550e8400-e29b-41d4-a716-446655440000

4.respusta exitosa :http 200 ok


```
{
  "status": "success",
  "message": "Cuadrilla consultada correctamente",
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Cuadrilla Norte",
    "isActive": true,
    "createdAt": "2026-11-05T13:00:00Z",
    "updatedAt": "2026-11-05T13:00:00Z",
    "drivers": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440001",
        "firstName": "Carlos",
        "lastName": "Pérez",
        "isActive": true
      }
    ],
    "crewMembers": [
      {
        "id": "550e8400-e29b-41d4-a716-446655440003",
        "fullName": "Juan Rodríguez",
        "isActive": true
      }
    ]
  }
}
```

5.respuesta de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

6. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada mediante la tabla session.

Comprobar que el rol del usuario tenga permisos para consultar cuadrillas.

Validar que el parámetro id tenga un formato UUID válido.

Comprobar que exista una cuadrilla con el identificador proporcionado.

Consultar los conductores relacionados mediante driver.crew_id.

Consultar los datos personales de los conductores mediante la relación driver.user_data_id.

Consultar los recolectores relacionados mediante crew_member.crew_id.

Devolver el estado actual de cada integrante según su correspondiente campo is_active.

7. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar la información detallada de una cuadrilla.

RN 02 Consulta de integrantes: el sistema debe mostrar el personal asociado a la cuadrilla.

RN 03 Identificación del personal: los conductores y recolectores deben mostrarse diferenciados según su tipo de registro.

RN 04 Estado del personal: el sistema debe identificar si cada integrante se encuentra activo o inactivo dentro de la cuadrilla.

RN 05 Integridad de la información: la consulta debe respetar las relaciones establecidas entre crew, driver, user_data y crew_member.

8. Relación con la base de datos

Este endpoint involucra cuatro tablas principales:

crew: almacena la información general de la cuadrilla.

driver: registra los conductores asociados.

user_data: proporciona los datos personales de los conductores.

crew_member: almacena los recolectores asociados.

Las relaciones principales son:

driver.crew_id → crew.id

driver.user_data_id → user_data.id

crew_member.crew_id → crew.id

9. Relación con las historias de usuario

HU5 Gestionar cuadrillas y turnos

Este endpoint permite consultar la composición de una cuadrilla e identificar el personal que la integra, cumpliendo con el criterio de mostrar los integrantes asociados a cada cuadrilla.

También contribuye a la organización del personal operativo para su posterior asignación a los turnos correspondientes.

Endpoint 23/ Actualizar cuadrilla

Método y ruta: PATCH /api/v1/crews/{id}

1. ¿Para qué sirve?

Permite actualizar parcialmente la información de una cuadrilla existente mediante su identificador UUID.

La operación permite modificar el nombre de la cuadrilla y cambiar su estado para activarla o desactivarla según las necesidades de la operación.

2. Parámetros de entrada

Parámetro de ruta: id

Descripción: identificador UUID de la cuadrilla que se desea actualizar.

Ejemplo de solicitud:

PATCH /api/v1/crews/550e8400-e29b-41d4-a716-446655440000

cuerpo de solicitud :JSON

```
{
  "name": "Cuadrilla Norte Actualizada",
  "isActive": false
}
```

Descripción de los campos:

Campo: name

Descripción: permite modificar el nombre de la cuadrilla. Se almacena en crew.name.

Campo: isActive

Descripción: permite cambiar el estado de la cuadrilla. Acepta valores booleanos true o false y se almacena en crew.is_active.

Ambos campos son opcionales, debido a que el método PATCH permite actualizar parcialmente la información.

3. Respuesta exitosa:http 200 ok

```
{
  "status": "success",
  "message": "Cuadrilla actualizada correctamente",
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "Cuadrilla Norte Actualizada",
    "isActive": false,
    "createdAt": "2026-11-05T13:00:00Z",
    "updatedAt": "2026-11-05T15:00:00Z"
  }
}
```

5. Respuesta de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

4. Validaciones

Antes de actualizar la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada mediante la tabla session.

Comprobar que el rol del usuario tenga permisos para actualizar cuadrillas.

Validar que el parámetro id tenga un formato UUID válido.

Comprobar que exista una cuadrilla con el identificador proporcionado.

Verificar que al menos uno de los campos permitidos haya sido enviado.

Validar que el nombre, cuando se envíe, no esté vacío ni contenga únicamente espacios en blanco.

Validar que isActive, cuando se envíe, contenga un valor booleano válido.

Actualizar el campo updated_at con la fecha y hora de la modificación.

5. Reglas de negocio

RN 01. Acceso según rol: solamente los usuarios autorizados pueden modificar la información de las cuadrillas.

RN 02. Actualización parcial: el sistema debe permitir modificar únicamente los campos enviados en la solicitud, conservando los demás valores.

RN 03. Activación y desactivación: una cuadrilla puede activarse o desactivarse según las necesidades de la operación.

RN 04. Estado de la cuadrilla: cuando una cuadrilla se desactive, debe quedar identificada como inactiva mediante is_active = false.

RN 05. Fecha de actualización: cada modificación debe actualizar el campo updated_at.

RN 06. Integridad de la información: la actualización no debe modificar ni eliminar los registros históricos del personal asociado a la cuadrilla.

6. Relación con la base de datos

Este endpoint utiliza la tabla:

crew: almacena y permite actualizar la información de la cuadrilla.

Campos involucrados:

id: identificador UUID de la cuadrilla.

name: nombre de la cuadrilla.

is_active: estado de la cuadrilla.

created_at: fecha de creación.

updated_at: fecha de actualización.

7. Relación con las historias de usuario

HU5 Gestionar cuadrillas y turnos

Este endpoint permite administrar las cuadrillas registradas, modificar su información y controlar su estado para facilitar la organización del personal operativo.

Contribuye al cumplimiento de los criterios de conformación y administración de cuadrillas establecidos en la historia de usuario.

TURNOS

Endpoint 24/crear turno

Método y ruta: POST /api/v1/schedules

1. ¿Para qué sirve?

Permite registrar un nuevo turno de trabajo en el sistema, definiendo su nombre, hora de inicio y hora de finalización.

Los turnos permiten organizar las jornadas de operación y posteriormente asociarlos a las microrrutas que serán atendidas durante determinados días de la semana.

2. Parámetros de entrada

Ejemplo del cuerpo de la solicitud (JSON):

{

"name": "Turno Mañana",

"startTime": "06:00:00",

"endTime": "12:00:00"

}

Descripción de los campos:

Campo: name

Descripción: nombre asignado al turno de trabajo. Se almacena en schedule.name.

Campo: startTime

Descripción: hora de inicio del turno. Se almacena en schedule.start_time y utiliza el tipo de dato TIME.

Campo: endTime

Descripción: hora de finalización del turno. Se almacena en schedule.end_time y utiliza el tipo de dato TIME.

3. Respuesta exitosa:http 201

{

"status": "success",

"message": "Turno creado correctamente",

"data": {

"id": "550e8400-e29b-41d4-a716-446655440010",

"name": "Turno Mañana",

"startTime": "06:00:00",

"endTime": "12:00:00",

"isActive": true,

"createdAt": "2026-11-05T13:00:00Z"

}

}

4. Respuestas de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

5. Validaciones

Antes de registrar la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada mediante la tabla session.

Comprobar que el rol del usuario tenga permisos para crear turnos.

Verificar que el campo name esté completo.

Validar que el nombre no contenga únicamente espacios en blanco.

Verificar que startTime y endTime estén presentes.

Validar que los horarios tengan un formato válido para el tipo de dato TIME.

Comprobar que el horario de inicio y finalización cumpla con las restricciones operativas definidas.

6. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden registrar turnos.

RN 02 Registro de turnos: cada turno debe contar con un nombre, una hora de inicio y una hora de finalización.

RN 03 Estado inicial: los turnos deben crearse activos (is_active = true), de acuerdo con el valor predeterminado de la tabla schedule.

RN 04 Identificación: cada turno debe contar con un identificador UUID generado automáticamente por la base de datos.

RN 05 Fecha de creación: el campo created_at registra automáticamente la fecha y hora de creación del turno.

RN 06 Horarios de operación: los horarios registrados deben respetar las restricciones operativas definidas para las jornadas de trabajo.

7. Relación con la base de datos

Este endpoint utiliza la tabla:

schedule: almacena la información de los turnos registrados.

Campos involucrados:

id: identificador UUID del turno.

name: nombre del turno.

start_time: hora de inicio.

end_time: hora de finalización.

is_active: estado del turno.

created_at: fecha de creación.

updated_at: fecha de actualización.

8. Relación con las historias de usuario

HU5 Gestionar cuadrillas y turnos

Este endpoint permite registrar los turnos de trabajo necesarios para organizar las jornadas operativas y posteriormente asociarlos a las microrrutas correspondientes.

Contribuye al cumplimiento del criterio que establece que el sistema debe permitir asociar una cuadrilla a un turno y organizar el personal disponible durante cada jornada.

Endpoint 25/listar turno

Método y ruta: GET /api/v1/schedules

1. ¿Para qué sirve?

Permite consultar el listado de turnos registrados en el sistema, mostrando su información general, horarios y estado actual.

La consulta facilita la administración de las jornadas de trabajo y permite identificar los turnos disponibles para la planificación operativa.

2. Parámetros de entrada

Este endpoint permite utilizar parámetros de consulta opcionales para filtrar y organizar los resultados.

Ejemplo de solicitud:

GET /api/v1/schedules?isActive=true&page=0&size=10

Descripción de los parámetros:

Campo: isActive

Descripción: permite filtrar los turnos según su estado. Acepta los valores true o false.

Campo: page

Descripción: indica el número de página que se desea consultar. Su valor predeterminado puede ser 0.

Campo: size

Descripción: establece la cantidad máxima de registros que se devolverán por página.

3. Respuesta exitosa : http 200 ok


```
{
 "status": "success",
 "message": "Turnos consultados correctamente",
 "data": [
  {
   "id": "550e8400-e29b-41d4-a716-446655440010",
   "name": "Turno Mañana",
   "startTime": "06:00:00",
   "endTime": "12:00:00",
   "isActive": true,
   "createdAt": "2026-11-05T13:00:00Z"
  },
  {
   "id": "550e8400-e29b-41d4-a716-446655440011",
   "name": "Turno Tarde",
```

```
    "startTime": "13:00:00",
    "endTime": "18:00:00",
    "isActive": false,
    "createdAt": "2026-11-05T14:00:00Z"
   }
 ],
 "pagination": {
   "page": 0,
   "size": 10,
   "totalElements": 2,
   "totalPages": 1
 }
}
```


4. Respuestas de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

5. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada mediante la tabla session.

Comprobar que el rol del usuario tenga permisos para consultar turnos.

Validar que el parámetro isActive, cuando se envíe, contenga un valor booleano válido.

Validar que los parámetros de paginación tengan valores permitidos.

Consultar los turnos registrados aplicando los filtros enviados.

6. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar los turnos.

RN 02 Consulta de turnos: el sistema debe permitir visualizar los turnos registrados.

RN 03 Identificación del estado: cada turno debe mostrar si se encuentra activo o inactivo.

RN 04 Filtrado: el sistema debe permitir consultar los turnos según su estado.

RN 05 Paginación: la consulta debe permitir dividir los resultados en páginas cuando la cantidad de registros lo requiera.

7. Relación con la base de datos

Este endpoint utiliza la tabla:

schedule: contiene los registros de los turnos.

Campos involucrados:

id

name

start_time

end_time

is_active

created_at

updated_at

8. Relación con las historias de usuario

HU5 Gestionar cuadrillas y turnos

Este endpoint permite consultar los turnos registrados para facilitar su administración y organización, identificando las jornadas disponibles para la planificación operativa.

Endpoint 26/Consultar turno por id

Método y ruta: GET /api/v1/schedules/{id}

1. ¿Para qué sirve?

Permite consultar la información detallada de un turno específico mediante su identificador UUID.

La consulta devuelve el nombre del turno, sus horarios y su estado actual, facilitando la identificación de la jornada de trabajo.

2. Tablas relacionadas

schedule: almacena la información general del turno.

3. Parámetros de entrada

Parámetro de ruta: id

Descripción: identificador UUID del turno que se desea consultar.

Ejemplo de solicitud:

GET /api/v1/schedules/550e8400-e29b-41d4-a716-446655440010

4. Respuesta exitosa


```
{
  "status": "success",
  "message": "Turno consultado correctamente",
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440010",
    "name": "Turno Mañana",
    "startTime": "06:00:00",
    "endTime": "12:00:00",
    "isActive": true,
    "createdAt": "2026-11-05T13:00:00Z",
    "updatedAt": "2026-11-05T13:00:00Z"
  }
}
```

5. Respuestas de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

6. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada mediante la tabla session.

Comprobar que el rol del usuario tenga permisos para consultar turnos.

Validar que el parámetro id tenga un formato UUID válido.

Comprobar que exista un turno con el identificador proporcionado.

Recuperar la información correspondiente al turno solicitado.

7. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar la información detallada de los turnos.

RN 02 Consulta individual: el sistema debe permitir consultar un turno específico mediante su identificador.

RN 03 Información del turno: la respuesta debe incluir su nombre, horario de inicio, horario de finalización y estado actual.

RN 04 Integridad de la información: la consulta debe devolver los datos correspondientes al registro existente en la tabla schedule.

8. Relación con la base de datos

Este endpoint utiliza la tabla:

schedule: contiene la información del turno solicitado.

Campos involucrados:

id

name

start_time

end_time

is_active

created_at

updated_at

9. Relación con las historias de usuario

HU5 Gestionar cuadrillas y turnos

Este endpoint permite consultar individualmente los turnos registrados, facilitando su administración y la organización de las jornadas de trabajo.

Endpoint 27/actualizar turno

Método y ruta: PATCH /api/v1/schedules/{id}

1. ¿Para qué sirve?

Permite actualizar parcialmente la información de un turno existente mediante su identificador UUID.

La operación permite modificar el nombre del turno, su hora de inicio, su hora de finalización y su estado, según las necesidades de organización de las jornadas operativas.

2. Tablas relacionadas

schedule: almacena la información de los turnos y permite actualizar los campos correspondientes a su nombre, horarios, estado y fecha de modificación.

3. Parámetros de entrada

Parámetro de ruta: id

Descripción: identificador UUID del turno que se desea actualizar.

Ejemplo de solicitud:

PATCH /api/v1/schedules/550e8400-e29b-41d4-a716-446655440010

cuerpo de solicitud :

{

"name": "Turno Mañana Actualizado",

"startTime": "05:30:00",

"endTime": "11:30:00",

"isActive": true

}

Descripción de los campos:

Campo: name

Descripción: permite modificar el nombre del turno. Se almacena en schedule.name.

Campo: startTime

Descripción: permite modificar la hora de inicio del turno. Se almacena en schedule.start_time.

Campo: endTime

Descripción: permite modificar la hora de finalización del turno. Se almacena en schedule.end_time.

Campo: isActive

Descripción: permite activar o desactivar el turno. Acepta valores booleanos true o false y se almacena en schedule.is_active.

Todos los campos del cuerpo de la solicitud son opcionales, debido a que el método PATCH permite actualizar parcialmente la información.

4. Respuesta exitosa: http 200 ok

{

"status": "success",

"message": "Turno actualizado correctamente",

"data": {

"id": "550e8400-e29b-41d4-a716-446655440010",

```
    "name": "Turno Mañana Actualizado",
    "startTime": "05:30:00",
    "endTime": "11:30:00",
    "isActive": true,
    "createdAt": "2026-11-05T13:00:00Z",
    "updatedAt": "2026-11-05T15:00:00Z"
  }
}
```


5. Respuestas de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

6. Validaciones

Antes de actualizar la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada mediante la tabla session.

Comprobar que el rol del usuario tenga permisos para actualizar turnos.

Validar que el parámetro id tenga un formato UUID válido.

Comprobar que exista un turno con el identificador proporcionado.

Verificar que al menos uno de los campos permitidos haya sido enviado.

Validar que el nombre, cuando se envíe, no esté vacío ni contenga únicamente espacios en blanco.

Validar que las horas enviadas tengan un formato válido para el tipo de dato TIME.

Comprobar que los horarios cumplan con las restricciones operativas establecidas.

Validar que isActive, cuando se envíe, contenga un valor booleano válido.

Actualizar el campo updated_at con la fecha y hora de la modificación.

7. Reglas de negocio

RN 01. Acceso según rol: solamente los usuarios autorizados pueden modificar la información de los turnos.

RN 02. Actualización parcial: el sistema debe permitir modificar únicamente los campos enviados en la solicitud, conservando los demás valores.

RN 03. Modificación de horarios: los horarios de inicio y finalización deben respetar las restricciones operativas definidas para las jornadas de trabajo.

RN 04. Activación y desactivación: el sistema debe permitir activar o desactivar un turno según las necesidades de la planificación.

RN 05. Estado del turno: cuando un turno se desactive, debe quedar identificado mediante is_active = false.

RN 06. Fecha de actualización: cada modificación debe actualizar el campo updated_at.

8. Relación con la base de datos

Este endpoint utiliza la tabla:

schedule: almacena y permite actualizar la información del turno.

Campos involucrados:

id: identificador UUID del turno.

name: nombre del turno.

start_time: hora de inicio.

end_time: hora de finalización.

is_active: estado del turno.

created_at: fecha de creación.

updated_at: fecha de actualización.

9. Relación con las historias de usuario

HU5 Gestionar cuadrillas y turnos

Este endpoint permite modificar la información de los turnos y controlar su estado para facilitar la organización de las jornadas de trabajo.

Contribuye al cumplimiento de los criterios relacionados con la gestión de turnos y la disponibilidad del personal operativo.

endpoints de macrorutas y microrutas

Endpoint 28/Listar macrorrutas

Método: GET Ruta: /api/v1/macroroutes

Autenticación: Bearer Token

Body: No requiere.

2. Para qué sirve?

Permite consultar las macrorrutas registradas, es decir, los sectores amplios que se atienden durante la operación.

El listado no incluye la geometría por defecto, debido a que esta información puede ser pesada y no es necesaria para llenar tablas o selectores.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El sistema debe verificar que el usuario autenticado tenga permisos para consultar las rutas.

La matriz de roles y permisos debe corresponder a lo establecido en HU1.

4.encabezado: http

Authorization: Bearer <access_token>

Accept: application/json

5. Parámetros de consulta

No requiere cuerpo de solicitud.

Descripción de los campos:

Campo: search

Tipo: string

Obligatorio: No.

Descripción: Busca macrorrutas por su nombre.

Campo: isActive

Tipo: boolean

Obligatorio: No.

Descripción: Filtra las macrorrutas según su estado. Por defecto devuelve únicamente las macrorrutas activas.

Campo: includeGeometry

Tipo: boolean

Obligatorio: No.

Descripción: Indica si se debe incluir la geometría de la macrorruta en formato GeoJSON. Por defecto es false.

Campo: page

Tipo: integer

Obligatorio: No.

Descripción: Número de página que se desea consultar. Por defecto es 1.

Campo: limit

Tipo: integer

Obligatorio: No.

Descripción: Cantidad de registros por página. Por defecto es 20 y permite un máximo de 100.

6. Respuesta exitosa

Código HTTP: 200 OK

{

"data": [

```
  {
    "id": "uuid-macrorruta",
    "name": "Macrorruta Norte",
    "microroutesCount": 4,
    "isActive": true,
    "createdAt": "2026-11-04T08:00:00Z"
  }
 ],
 "meta": {
  "page": 1,
  "limit": 20,
  "total": 1
 }
}
```


7. Respuestas de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada.

Comprobar que el rol del usuario tenga permiso para consultar rutas.

Validar los parámetros de consulta y la paginación.

Consultar la tabla macroroute.

Contar las microrrutas activas asociadas a cada macrorruta.

Aplicar los filtros y la paginación solicitados.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar rutas.

Se deben poder consultar las macrorrutas registradas, según HU7.

Por defecto, solo se devuelven las macrorrutas activas.

La geometría únicamente se devuelve cuando se solicita explícitamente mediante includeGeometry=true.

10. Relación con la base de datos

Este endpoint involucra principalmente dos tablas:

macroroute: contiene las macrorrutas y su geometría.

microroute: se utiliza para contar las microrrutas asociadas.

la relacion principal es:

microroute.macroroute_id → macroroute.id

11. Relación con las historias de usuario

HU7 Gestionar macrorrutas

Este endpoint cumple el criterio de permitir consultar las macrorrutas registradas.

Endpoint 29/Crear macrorruta

Método: POST Ruta: /api/v1/macroroutes

Autenticación: Bearer Token

Body: Sí requiere.

2. Para qué sirve?

Permite que el jefe de operaciones cree una macrorruta para organizar los sectores que serán atendidos durante la operación.

La macrorruta se crea inicialmente sin microrrutas. Estas se crean posteriormente mediante el Endpoint 32 y quedan asociadas a ella.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

Solamente podrán utilizar este endpoint los roles autorizados para gestionar rutas.

La matriz de roles y permisos está pendiente de definir en HU1.

4.encabezado http:

Authorization: Bearer <access_token>

Content-Type: application/json

Accept: application/json

5.cuerpo de solicitud


```
{
  "name": "Macrorruta Norte",
  "geometry": {
    "type": "MultiPolygon",
    "coordinates": [
      [
        [
          [-74.08, 4.70],
          [-74.07, 4.70],
          [-74.07, 4.71],
          [-74.08, 4.71],
          [-74.08, 4.70]
        ]
      ]
    ]
  }
}
```

Descripción de los campos:

Campo: name

Tipo: string (máximo 255 caracteres)

Obligatorio: Sí.

Descripción: Nombre de la macrorruta.

Campo: geometry

Tipo: objeto GeoJSON (MultiPolygon)

Obligatorio: Pendiente de definir.

Descripción: Área geográfica que cubre la macrorruta, representada mediante coordenadas de longitud y latitud (SRID 4326).

HU7 establece que cada macrorruta debe contar con la información requerida, pero aún no se ha definido si la geometría es obligatoria al momento de crearla.

6. Respuesta exitosa

Código HTTP: 201 Created


```
{
  "id": "uuid-macrorruta",
  "name": "Macrorruta Norte",
  "geometry": {
    "type": "MultiPolygon",
    "coordinates": [
      [
        [
          [-74.08, 4.70],
          [-74.07, 4.70],
          [-74.07, 4.71],
          [-74.08, 4.71],
```

```
        [-74.08, 4.70]
      ]
    ]
  ]
 },
 "microroutesCount": 0,
 "isActive": true,
 "createdAt": "2026-11-04T08:00:00Z"
}
```


7. Respuestas de error

400 Bad Request: Faltan campos obligatorios, la geometría no es un MultiPolygon válido o contiene coordenadas fuera de rango.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para crear rutas.

409 Conflict: Ya existe una macrorruta activa con el mismo nombre, si se aprueba esa restricción.

500 Internal Server Error: Ocurrió un error inesperado durante el registro.

8. Validaciones

Antes de crear la macrorruta, el backend debe:

Validar la sesión y comprobar que el rol tenga permiso para gestionar rutas.

Verificar que se haya enviado el nombre y que no esté vacío.

Validar que el nombre sea único entre las macrorrutas activas, pendiente de confirmar.

Validar que la geometría sea un GeoJSON de tipo MultiPolygon, con coordenadas válidas.

Verificar que la longitud esté entre -180 y 180 y la latitud entre -90 y 90.

Validar que la geometría sea topológicamente válida, sin autointersecciones.

Convertir la geometría al SRID 4326 antes de guardarla.

Crear la macrorruta como activa.

Registrar la creación en audit_log, incluyendo el actor, la acción, la entidad y los valores nuevos.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los roles autorizados pueden crear rutas.

Se debe poder crear una macrorruta, según HU7.

Cada macrorruta debe contar con la información requerida.

Una macrorruta debe poder relacionarse con sus microrrutas.

Pendiente por definir: establecer si se permite que dos macrorrutas se solapen geográficamente.

10. Relación con la base de datos

Este endpoint involucra principalmente dos tablas:

macroroute: se crea el registro de la macrorruta con los campos name, geom, is_active, created_at y updated_at.

audit_log: registra la creación de la macrorruta.

La relación principal es:

microroute.macroroute_id → macroroute.id

11. Relación con las historias de usuario

HU7 Gestionar macrorrutas

Este endpoint cumple los siguientes criterios:

Se debe poder crear una macrorruta.

Cada macrorruta debe contar con la información requerida.

endpoint 30/Consultar macrorruta

Método: GET Ruta: /api/v1/macroroutes/{id}

Autenticación: Bearer Token

Body: No requiere.

2. Para qué sirve?

Permite consultar el detalle de una macrorruta, incluyendo su geometría y las microrrutas asociadas.

Se utiliza para mostrar la macrorruta en el visor de mapa y consultar los recorridos que la componen.

Relacionado con: HU7 — Gestionar macrorrutas.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El rol del usuario debe tener permisos para consultar rutas.

4. Encabezados HTTP

Authorization: Bearer <access_token>

Accept: application/json

5. Parámetros de ruta

No requiere cuerpo de solicitud.

Descripción de los campos:

Campo: id

Tipo: string (UUID)

Obligatorio: Sí.

Descripción: Identificador único de la macrorruta que se desea consultar.

6. Respuesta exitosa

Código HTTP: 200 OK


```
{
  "id": "uuid-macrorruta",
  "name": "Macrorruta Norte",
```

```
 "geometry": {
  "type": "MultiPolygon",
  "coordinates": [
   [
    [
     [-74.08, 4.70],
     [-74.07, 4.70],
     [-74.07, 4.71],
     [-74.08, 4.71],
     [-74.08, 4.70]
    ]
   ]
  ]
 },
 "microroutes": [
  {
   "id": "uuid-microrruta",
   "name": "Microrruta N-01",
   "isActive": true
  }
 ],
 "isActive": true,
 "createdAt": "2026-11-04T08:00:00Z",
 "updatedAt": "2026-11-04T08:00:00Z"
}
```


Consideración: el arreglo microroutes devuelve únicamente un resumen de cada microrruta: identificador, nombre y estado. El detalle de cada una se consulta mediante el Endpoint 33.

7. Respuestas de error

400 Bad Request: El identificador no tiene un formato UUID válido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

404 Not Found: No existe una macrorruta con el identificador enviado.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de devolver la información, el backend debe:

Validar la sesión y comprobar que el rol tenga permiso para consultar rutas.

Validar que el parámetro id tenga formato UUID.

Buscar la macrorruta en la tabla macroroute.

Obtener las microrrutas asociadas.

Convertir la geometría a formato GeoJSON.

Responder con código 404 si la macrorruta no existe.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar rutas.

Una macrorruta debe poder relacionarse con sus microrrutas, según HU7.

Una macrorruta inactiva puede consultarse, pero el sistema debe indicar que se encuentra inactiva.

10. Relación con la base de datos

Este endpoint involucra principalmente dos tablas:

macroroute: contiene los datos y la geometría de la macrorruta.

microroute: permite consultar las microrrutas asociadas.

La relación principal es:

microroute.macroroute_id → macroroute.id

11. Relación con las historias de usuario

HU7 Gestionar macrorrutas

Este endpoint cumple los siguientes criterios:

Se deben poder consultar las macrorrutas registradas.

Una macrorruta debe poder relacionarse con sus microrrutas.

Endpoint 31/Listar microrrutas

Método: GET Ruta: /api/v1/microroutes

Módulo: Microrrutas

Autenticación: Bearer Token

Body: No requiere.

2. Para qué sirve?

Permite consultar las microrrutas registradas, es decir, los recorridos específicos de atención.

También permite filtrarlas por la macrorruta a la que pertenecen.

Esta consulta sirve como base para seleccionar una microrruta al momento de asignar recursos, según HU10.

Relacionado con: HU8 Crear microrrutas.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El rol del usuario debe tener permisos para consultar rutas.

4. Encabezados HTTP

Authorization: Bearer <access_token>

Accept: application/json

5. Parámetros de consulta

No requiere cuerpo de solicitud.

Descripción de los campos:

Campo: macrorouteId

Tipo: string (UUID)

Obligatorio: No.

Descripción: Filtra las microrrutas pertenecientes a una macrorruta específica.

Campo: search

Tipo: string

Obligatorio: No.

Descripción: Busca microrrutas por su nombre.

Campo: isActive

Tipo: boolean

Obligatorio: No.

Descripción: Filtra las microrrutas según su estado. Por defecto devuelve únicamente las microrrutas activas.

Campo: includeGeometry

Tipo: boolean

Obligatorio: No.

Descripción: Indica si se debe incluir la geometría de la microrruta en formato GeoJSON. Por defecto es false.

Campo: page

Tipo: integer

Obligatorio: No.

Descripción: Número de página que se desea consultar. Por defecto es 1.

Campo: limit

Tipo: integer

Obligatorio: No.

Descripción: Cantidad de registros por página. Por defecto es 20 y permite un máximo de 100.

6. Respuesta exitosa

Código HTTP: 200 OK


```
{
  "data": [
    {
      "id": "uuid-microrruta",
      "name": "Microrruta N-01",
      "macroroute": {
        "id": "uuid-macrorruta",
        "name": "Macrorruta Norte"
      },
      "neighborhoodsCount": 5,
      "isActive": true
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1
  }
}
```


7. Respuestas de error

400 Bad Request: Algún parámetro de consulta tiene un formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de devolver la información, el backend debe:

Verificar que se haya enviado el token de autenticación.

Validar que corresponda a una sesión existente, no revocada y no expirada.

Comprobar que el rol del usuario tenga permiso para consultar rutas.

Validar que macrorouteId tenga formato UUID cuando se envíe.

Validar los parámetros de consulta y la paginación.

Consultar la tabla microroute mediante una relación con macroroute.

Contar los barrios asociados a cada microrruta.

Aplicar los filtros y la paginación solicitados.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar rutas.

Se deben poder consultar las microrrutas creadas, según HU8.

Por defecto, solo se devuelven las microrrutas activas.

La geometría únicamente se devuelve cuando se solicita explícitamente mediante includeGeometry=true.

10. Relación con la base de datos

Este endpoint involucra principalmente tres tablas:

microroute: contiene las microrrutas y su geometría.

macroroute: contiene la macrorruta a la que pertenece cada microrruta.

microrute_neighborhood: se utiliza para contar los barrios asociados.

Las relaciones principales son:

microroute.macroroute_id → macroroute.id

microrute_neighborhood.microroute_id → microroute.id

11. Relación con las historias de usuario

HU8 Crear microrrutas

Este endpoint cumple el criterio de permitir consultar las microrrutas creadas.

endpoint 32/Crear microrruta

Método: POST

Ruta: /api/v1/microroutes

Módulo: Microrrutas

Autenticación: Bearer Token

Body: Sí requiere.

2. Para qué sirve?

Permite que el jefe de operaciones cree una microrruta asociada a una macrorruta para definir un recorrido específico de atención.

Opcionalmente, se pueden indicar los horarios en los que opera la microrruta.

Los barrios que atenderá se asocian posteriormente mediante el endpoint correspondiente al módulo de Barrios.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

Solamente podrán utilizar este endpoint los roles autorizados para gestionar rutas.

4. Encabezados HTTP

Authorization: Bearer <access_token>

Content-Type: application/json

Accept: application/json

5. Cuerpo de la solicitud


```
{
  "macrorouteId": "uuid-macrorruta",
  "name": "Microrruta N-01",
  "geometry": {
    "type": "MultiLineString",
    "coordinates": [
      [
        [-74.080, 4.700],
        [-74.075, 4.703],
        [-74.070, 4.705]
      ]
    ]
  },
  "schedules": [
    {
      "scheduleId": "uuid-turno",
      "dayOfWeek": 1
    }
  ]
}
```


Descripción de los campos:

Campo: macrorouteId

Tipo: string (UUID)

Obligatorio: Sí.

Descripción: Identificador de la macrorruta a la que pertenece la microrruta. La macrorruta debe existir y estar activa.

Campo: name

Tipo: string (máximo 255 caracteres)

Obligatorio: Sí.

Descripción: Nombre de la microrruta.

Campo: geometry

Tipo: objeto GeoJSON (MultiLineString)

Obligatorio: Pendiente de definir.

Descripción: Trazado del recorrido, representado mediante coordenadas de longitud y latitud (SRID 4326). HU8 solicita registrar la información necesaria del recorrido, pero aún no se ha definido si la geometría es obligatoria al crear la microrruta.

Campo: schedules

Tipo: array

Obligatorio: No.

Descripción: Horarios en los que opera la microrruta. Está pendiente definir si se envían en este endpoint o mediante un endpoint propio.

Campo: schedules[].scheduleId

Tipo: string (UUID)

Obligatorio: Sí, dentro de cada elemento del arreglo.

Descripción: Identificador del horario perteneciente al catálogo schedule.

Campo: schedules[].dayOfWeek

Tipo: integer

Obligatorio: Sí, dentro de cada elemento del arreglo.

Descripción: Día de la semana en el que opera la microrruta. Está pendiente definir la convención. Se propone utilizar 1 para lunes y 7 para domingo, siguiendo el formato ISO.

6. Respuesta exitosa

Código HTTP: 201 Created

Devuelve la microrruta creada con la misma estructura de respuesta del Endpoint 33 — Consultar microrruta.

7. Respuestas de error

400 Bad Request: Faltan campos obligatorios, la geometría no es un MultiLineString válido, el día de la semana está fuera de rango o existen horarios repetidos.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para crear rutas.

404 Not Found: La macrorruta o alguno de los horarios enviados no existen.

409 Conflict: Ya existe una microrruta activa con el mismo nombre dentro de la macrorruta, si se aprueba esa restricción.

500 Internal Server Error: Ocurrió un error inesperado durante el registro.

8. Validaciones

Antes de crear la microrruta, el backend debe:

Validar la sesión y comprobar que el rol tenga permiso para gestionar rutas.

Verificar que se hayan enviado los campos macrorouteId y name.

Comprobar que la macrorruta exista y esté activa.

Validar que el nombre sea único dentro de la macrorruta, pendiente de confirmar.

Validar que la geometría sea un GeoJSON de tipo MultiLineString con coordenadas válidas.

Validar que el trazado quede dentro del área de la macrorruta. Está pendiente definir si esta validación será obligatoria o solamente una advertencia.

Comprobar que cada scheduleId exista y esté activo.

Validar que dayOfWeek esté dentro del rango establecido.

Verificar que no se repita la misma combinación de horario y día.

Crear la microrruta y sus horarios en una misma transacción de base de datos.

Registrar la creación en audit_log.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los roles autorizados pueden crear rutas.

RN08 Integridad de datos: la macrorruta y los horarios deben corresponder a registros existentes.

Una microrruta debe estar asociada a una macrorruta, según HU8.

Se debe poder registrar la información necesaria del recorrido, según HU8.

Una microrruta debe poder contener los barrios que serán atendidos, según HU8. Los barrios se asocian mediante el endpoint correspondiente al módulo de Barrios.

Pendiente por definir:

Establecer si una microrruta puede crearse sin horarios.

Definir si el horario forma parte de la definición de una microrruta o si se administra mediante endpoints independientes.

10. Relación con la base de datos

Este endpoint involucra principalmente cuatro tablas:

microroute: permite crear el registro de la microrruta.

macroroute: permite verificar la macrorruta a la que se asociará.

microrute_schedule: permite crear las relaciones entre la microrruta y sus horarios, utilizando microroute_id, schedule_id y day_of_week.

audit_log: registra la creación de la microrruta.

Las relaciones principales son:

microroute.macroroute_id → macroroute.id

microrute_schedule.microroute_id → microroute.id

microrute_schedule.schedule_id → schedule.id

Se recomienda establecer una restricción UNIQUE sobre la combinación (microroute_id, schedule_id, day_of_week) y una restricción CHECK que limite day_of_week al rango que se defina.

11. Relación con las historias de usuario

HU8 Crear microrrutas

Este endpoint cumple los siguientes criterios:

Una microrruta debe estar asociada a una macrorruta.

Se debe poder registrar la información necesaria del recorrido.

HU9 Asociar barrios a una microrruta

La creación permite establecer la microrruta a la que posteriormente se asociarán los barrios que atenderá.

Endpoint 33/Consultar microrruta

Método: GET

Ruta: /api/v1/microroutes/{id}

Autenticación: Bearer Token

Body: No requiere.

2. Para qué sirve?

Permite consultar el detalle de una microrruta, incluyendo:

La macrorruta a la que pertenece.

El trazado del recorrido.

Los barrios que atiende, organizados por orden de visita.

Los horarios en los que opera.

Se utiliza para mostrar la microrruta en el visor de mapa y verificar qué sectores atenderá.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El rol del usuario debe tener permisos para consultar rutas.

4. Encabezados HTTP

Authorization: Bearer <access_token>

Accept: application/json

5. Parámetros de ruta

No requiere cuerpo de solicitud.

Descripción de los campos:

Campo: id

Tipo: string (UUID)

Obligatorio: Sí.

Descripción: Identificador único de la microrruta que se desea consultar.

6. Respuesta exitosa

Código HTTP: 200 OK


```
{
  "id": "uuid-microrruta",
  "name": "Microrruta N-01",
  "macroroute": {
    "id": "uuid-macrorruta",
    "name": "Macrorruta Norte"
  },
  "geometry": {
    "type": "MultiLineString",
    "coordinates": [
      [
        [-74.080, 4.700],
        [-74.075, 4.703],
        [-74.070, 4.705]
```

```
    ]
   ]
  },
  "neighborhoods": [
   {
    "id": "uuid-barrio",
    "name": "Barrio Ejemplo",
    "commune": "Comuna 1",
    "orderVisit": 1
   }
  ],
  "schedules": [
   {
    "id": "uuid-microroute-schedule",
    "dayOfWeek": 1,
    "schedule": {
     "id": "uuid-turno",
     "name": "Turno mañana",
     "startTime": "06:00",
     "endTime": "14:00"
    },
    "isActive": true
   }
  ],
  "isActive": true,
  "createdAt": "2026-11-04T08:00:00Z",
  "updatedAt": "2026-11-04T08:00:00Z"
}
```

7. Respuestas de error

400 Bad Request: El identificador no tiene un formato UUID válido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

404 Not Found: No existe una microrruta con el identificador enviado.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de devolver la información, el backend debe:

Validar la sesión y comprobar que el rol tenga permiso para consultar rutas.

Validar que el parámetro id tenga formato UUID.

Buscar la microrruta en la tabla microroute, incluyendo su macrorruta asociada.

Obtener los barrios asociados, ordenados por order_visit.

Obtener los horarios activos asociados a la microrruta.

Convertir la geometría a formato GeoJSON.

Responder con código 404 si la microrruta no existe.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar rutas.

La microrruta debe permitir consultar los sectores que atenderá, según HU9.

Una microrruta inactiva puede consultarse, pero el sistema debe indicar que se encuentra inactiva.

10. Relación con la base de datos

Este endpoint involucra principalmente cinco tablas:

microroute: contiene los datos y el trazado de la microrruta.

macroroute: contiene la macrorruta a la que pertenece.

microrute_neighborhood: contiene los barrios asociados y su orden de visita.

btura_neighborhood: contiene el nombre y la comuna de cada barrio.

microrute_schedule y schedule: permiten consultar los horarios de operación.

Las relaciones principales son:

microroute.macroroute_id → macroroute.id

microrute_neighborhood.microroute_id → microroute.id

microrute_neighborhood.neighborhood_id → btura_neighborhood.id

microrute_schedule.microroute_id → microroute.id

microrute_schedule.schedule_id → schedule.id

11. Relación con las historias de usuario

HU8 Crear microrrutas

Permite consultar la información de la microrruta creada.

HU9 Asociar barrios a una microrruta

Permite consultar los barrios asociados a la microrruta y el orden en el que deben atenderse.

Endpoint 34/Consultar barrios

Método y ruta: GET /api/v1/neighborhoods

Autenticación: Bearer Token (JWT).

Body: No requiere.

1. ¿Para qué sirve?

Permite consultar los barrios registrados en VINTRA. La información se utiliza para identificar los barrios que pueden asociarse a las microrrutas.

2. Parametros

parametro:name

tipo:string

descripcion:Buscar barrios por nombre.

parametro:commune

tipo:string

descripcion:Filtrar barrios por comuna.

parametro:page

tipo:integer

descripcion:Número de página.

parametro:limit

tipo:integer

descripcion:Cantidad de registros por página.

encabezado: http

Authorization: Bearer <access_token>

Accept: application/json

3. Respuesta exitosa — 200 OK


```
{
  "data": [
    {
      "id": "uuid-barrio-1",
      "name": "El Cristal",
      "commune": "Comuna 8"
    },
    {
```

```
    "id": "uuid-barrio-2",
    "name": "La Independencia",
    "commune": "Comuna 10"
   }
 ],
 "meta": {
   "page": 1,
   "limit": 10,
   "total": 2
 }
}
```


4.posibles errores

401 Unauthorized : No se proporcionó un token válido.

403 Forbidden :El usuario no tiene permisos para consultar los barrios.

500 Internal Server Error :Ocurrió un error inesperado.

5. Validaciones

Verificar que los filtros tengan un formato válido.

Comprobar que page y limit sean valores positivos.

Aplicar los filtros sobre los campos existentes en la tabla.

6. Reglas de negocio

La consulta debe mostrar únicamente barrios registrados.

Los filtros deben aplicarse sobre el nombre y la comuna.

La consulta no debe modificar la información almacenada.

relacion con las historias de usuario:

1. Relación con las historias de usuario

HU9 Asociar barrios a una microrruta

Este endpoint permite cumplir los siguientes criterios:

Se deben poder consultar los barrios registrados.

El sistema debe permitir consultar los barrios disponibles para asociarlos a una microrruta.

7. Relación con la base de datos

Tabla: btura_neighborhood.:contiene la información registrada de los barrios, como su identificador, nombre, comuna y ubicación geográfica.

Campos utilizados:

id

name

commune

geom

created_at

No requiere relaciones adicionales para consultar el catálogo de barrios.

Endpoint 35/Asociar barrio a microrruta

Método y ruta: POST /api/v1/microroutes/{id}/neighborhoods

Autenticación: Bearer Token (JWT).

¿Para qué sirve?

Permite asociar un barrio a una microrruta existente, registrando también el orden en el que debe visitarse dentro del recorrido

parametro: id

tipo:uuid

obligatorio: si

descripción:Identificador de la microrruta a la que se asociará el barrio.

2. Encabezado: http

Authorization: Bearer <access_token>

Content-Type: application/json

Accept: application/json

3. Cuerpo de la solicitud

{

"neighborhoodId": "uuid-del-barrio",

"orderVisit": 1

}

campo: neighborhoodId

tipo:UUID

obligatorio:si

descricion:Identificador del barrio que se asociará.

campo: orderVisit

tipo: integer

obligatorio: si

descricion: Orden de visita del barrio dentro de la microrruta.

4.Respuesta exitosa — 201 Created

{

"id": "uuid-asociacion",

"microrouteId": "uuid-microrruta",

"neighborhoodId": "uuid-barrio",

"orderVisit": 1

}

5.posibles errores

400 Bad Request: Los datos enviados son inválidos o faltan campos obligatorios.

401 Unauthorized :No se proporcionó un token válido.

404 Not Found :La microrruta o el barrio no existen.

409 Conflict :El barrio ya está asociado a la microrruta o existe un conflicto con el orden de visita, si esa regla se establece.

500 Internal Server Error :Ocurrió un error inesperado.

6. Validaciones

Verificar que el identificador de la microrruta sea válido.

Comprobar que la microrruta exista.

Verificar que el barrio exista en btura_neighborhood.

Comprobar que orderVisit sea un entero positivo.

Verificar que no se esté registrando una asociación duplicada.

Comprobar que la microrruta esté activa, si la regla de negocio exige que solo se puedan modificar microrrutas activas.

7. Reglas de negocio

Un barrio puede asociarse a una microrruta mediante la tabla de relación.

Cada asociación debe registrar su orden de visita.

El orden de visita permite establecer la secuencia de los barrios dentro del recorrido.

No se deben registrar asociaciones duplicadas.

8.Relación con historias de usuario :

Este endpoint permite cumplir los siguientes criterios:

Se deben poder asociar barrios registrados a una microrruta.

El sistema debe mostrar los barrios asociados.

La microrruta debe permitir consultar los sectores que atenderá.

Se debe conservar el orden de visita de los barrios dentro del recorrido

9. Relación con la base de datos

Tablas involucradas:

btura_neighborhood :contiene la información de los barrios registrados.

microroute :contiene la información de las microrrutas a las que se asociarán los barrios.

microrute_neighborhood:establece la relación entre las microrrutas y los barrios, almacenando también el orden de visita

relaciones:

microrute_neighborhood.neighborhood_id

→ btura_neighborhood.id

microrute_neighborhood.microroute_id

→ microroute.id

Asignaciones

Endpoint 36/Consultar asignaciones

Método y ruta: GET /api/v1/assignments

Autenticación: Bearer Token (JWT).

Body: No requiere.

2.¿Para qué sirve?

Permite consultar las asignaciones registradas en VINTRA, mostrando la información de las cuadrillas, microrrutas, vehículos, estados y fechas programadas.

1. Parámetros

parametro:crewId

tipo:UUID

descripcion:Filtrar por cuadrilla.

parametro:microrouteId

tipo:UUID

descripcion:Filtrar por microrruta.

parametro:vehicleId

tipo:UUID

descripcion:Filtrar por vehículo.

parametro:assignmentStatusId

tipo:UUID

descripcion:Filtrar por estado de asignación.

parametro:scheduledDate

tipo:date

descripcion:Filtrar por fecha programada.

parametro:page

tipo:integer

descripcion:Número de página.

parametro:limit

tipo:integer

descripcion:Cantidad de registros por página.

3.encabezado:

Authorization: Bearer <access_token>

Accept: application/json

respuesta exitosa 200 ok

```
{
  "data": [
    {
      "id": "uuid-asignacion",
      "crew": {
        "id": "uuid-cuadrilla",
        "name": "Cuadrilla 01"
      },
      "microroute": {
        "id": "uuid-microrruta",
        "name": "Microrruta Norte"
      },
      "vehicle": {
        "id": "uuid-vehiculo",
        "plate": "ABC123"
      },
      "assignmentStatus": {
        "id": "uuid-estado",
        "code": "PENDING",
        "name": "Pendiente"
      },
      "scheduledDate": "2026-11-04",
      "startedAt": null,
      "endedAt": null,
      "createdAt": "2026-11-04T08:00:00Z",
      "updatedAt": "2026-11-04T08:00:00Z"
    }
  ],
```

```
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1
  }
}
```


4.posibles errores

400 Bad Request: Algún filtro tiene un formato inválido.

401 Unauthorized :o se proporcionó un token válido.

403 Forbidden :El usuario no tiene permisos para consultar las asignaciones.

500 Internal Server Error :Ocurrió un error inesperado.

5. Validaciones

Verificar el formato de los identificadores enviados como filtros.

Comprobar que scheduledDate tenga formato de fecha válido.

Validar que page y limit sean valores positivos.

Consultar las relaciones correspondientes para mostrar los datos asociados.

6. Reglas de negocio

La consulta debe mostrar las asignaciones registradas.

Los filtros deben permitir localizar asignaciones por cuadrilla, microrruta, vehículo, estado o fecha.

La consulta no debe modificar las asignaciones.

6.Relacion con las historias de usuario:

HU10 Asignar recursos a una microrruta

Este endpoint permite cumplir los siguientes criterios:

Se deben poder consultar las asignaciones realizadas.

Se debe identificar el vehículo, la cuadrilla y la microrruta asociados.

Se debe conocer el estado de cada asignación.

Se debe facilitar la identificación de los recursos que participan en la operación.

HU12 Trazabilidad de asignaciones

También contribuye a este requisito:

Permite consultar las asignaciones sobre las cuales se necesita revisar la trazabilidad de los cambios.

7. Relación con la base de datos

Tablas involucradas:

assignment:contiene la información registrada de las asignaciones, como su identificador, fecha programada, fecha de inicio, fecha de finalización y referencias a los recursos asignados.

crew:contiene la información de las cuadrillas.

microroute:contiene la información de las microrrutas.

vehicle: contiene la información de los vehículos asignados.

assignment_status :permite consultar el estado de cada asignación.

relaciones:

assignment.crew_id → crew.id

assignment.microroute_id → microroute.id

assignment.vehicle_id → vehicle.id

assignment.assignment_status_id → assignment_status.id

Endpoint 37/Consultar una asignación

Método y ruta: GET /api/v1/assignments/{id}

Autenticación: Bearer Token (JWT).

Body: No requiere.

¿Para qué sirve?

Permite consultar el detalle de una asignación específica, identificada mediante su ID.

Su finalidad es mostrar la información completa del recorrido programado, incluyendo la cuadrilla responsable, la microrruta, el vehículo asignado, su estado y las fechas correspondientes.

parametros:

parametro:id

tipo:uuid

obligatorio:si

descripcion:Identificador de la asignación.

encabezado:

Authorization: Bearer <access_token>

Accept: application/json

3.respuesta exitosa-200 ok


```
{
  "id": "uuid-asignacion",
  "crew": {
    "id": "uuid-cuadrilla",
    "name": "Cuadrilla 01"
  },
  "microroute": {
    "id": "uuid-microrruta",
    "name": "Microrruta Norte"
  },
  "vehicle": {
    "id": "uuid-vehiculo",
```

```
    "plate": "ABC123"
  },
  "assignmentStatus": {
    "id": "uuid-estado",
    "code": "IN_PROGRESS",
    "name": "En ejecución"
  },
  "scheduledDate": "2026-11-04",
  "startedAt": "2026-11-04T08:00:00Z",
  "endedAt": null,
  "createdAt": "2026-11-03T15:00:00Z",
  "updatedAt": "2026-11-04T08:00:00Z"
}
```


4.posibles errores

401 Unauthorized :No se proporcionó un token válido.

403 Forbidden : El usuario no tiene permisos para consultar la asignación.

404 Not Found:No existe una asignación con el identificador proporcionado.

500 Internal Server Error :Ocurrió un error inesperado

5. Validaciones

Verificar que el identificador tenga formato UUID.

Comprobar que la asignación exista.

Consultar los datos relacionados de la cuadrilla, microrruta, vehículo y estado.

6. Reglas de negocio

La información debe corresponder a la asignación solicitada.

El estado debe obtenerse del catálogo assignment_status.

Las fechas de inicio y finalización deben reflejar los valores almacenados en la asignación.

6. Relación con las historias de usuario

HU10 Asignar recursos a una microrruta

Este endpoint permite cumplir los siguientes criterios:

Se debe poder consultar la información de una asignación específica.

Se debe identificar la microrruta seleccionada.

Se deben consultar los recursos asociados: vehículo y cuadrilla.

Se debe conocer el estado de la asignación.

HU12 Trazabilidad de asignaciones

También contribuye a este requisito:

Permite identificar la asignación específica cuyo historial de modificaciones se desea consultar.

7. Relación con la base de datos

Tablas involucradas:

assignment :contiene los datos principales de la asignación, como su identificador, fecha programada, fecha de inicio y fecha de finalización

crew:permite consultar la cuadrilla relacionada con la asignación.

microroute:permite consultar la microrruta asignada.

vehicle:permite consultar el vehículo asociado.

assignment_status:permite obtener el código y nombre del estado de la asignación.

Endpoint 38/Crear asignación

Método y ruta: POST /api/v1/assignments

Autenticación: Bearer Token (JWT).

1.¿Para qué sirve?

Permite registrar una nueva asignación en VINTRA, relacionando una cuadrilla, una microrruta y un vehículo, además de establecer el estado y la fecha programada para realizar el recorrido.

2. Encabezados

Authorization: Bearer <access_token>

Content-Type: application/json

Accept: application/json

cuerpo de la solicitud

```
{
  "crewId": "uuid-cuadrilla",
  "microrouteId": "uuid-microrruta",
  "vehicleId": "uuid-vehiculo",
  "assignmentStatusId": "uuid-estado",
  "scheduledDate": "2026-11-05"
}
```

campo: crewId

tipo:UUID

obligatorio:si

descripcion:Identificador de la cuadrilla asignada.

campo:microrouteId

tipo:UUID

obligatorio:si

descripcion:Identificador de la microrruta.

campo:vehicleId

tipo:UUID

obligatorio:si

descripcion:Identificador del vehículo asignado.

campo:assignmentStatusId

tipo :UUID

obligatorio:si

descripcion:Identificador del estado de la asignación.

campo:scheduledDate

tipo: date

obligatorio:si

descripcion:Fecha programada para realizar el recorrido.

3.respuesta exitosa 201


```
{
  "id": "uuid-asignacion",
  "crewId": "uuid-cuadrilla",
  "microrouteId": "uuid-microrruta",
  "vehicleId": "uuid-vehiculo",
  "assignmentStatus": {
    "id": "uuid-estado",
    "code": "PENDING",
    "name": "Pendiente"
  },
  "scheduledDate": "2026-11-05",
  "startedAt": null,
  "endedAt": null,
  "createdAt": "2026-11-04T10:00:00Z",
  "updatedAt": "2026-11-04T10:00:00Z"
}
```

4.posibles errores

400 Bad Request :Faltan campos obligatorios o los datos enviados son inválidos.

401 Unauthorized :No se proporcionó un token válido

403 Forbidden :El usuario no tiene permisos para crear asignaciones.

404 Not Found :La cuadrilla, microrruta, vehículo o estado no existen.

409 Conflict :Existe un conflicto con la disponibilidad de los recursos asignados, si se establece esa regla.

500 Internal Server Error :Ocurrió un error inesperado.

5. Validaciones

Verificar que se hayan enviado todos los campos obligatorios.

Comprobar que la cuadrilla exista.

Verificar que la microrruta exista y esté activa.

Comprobar que el vehículo exista y esté activo.

Verificar que el estado exista y esté activo.

Validar el formato de scheduledDate.

Comprobar la disponibilidad de la cuadrilla y del vehículo para la fecha programada, de acuerdo con las reglas de asignación.

Verificar que la fecha programada sea compatible con los horarios definidos para la microrruta en microroute_schedule.

6. Reglas de negocio

Toda asignación debe estar asociada a una cuadrilla, una microrruta y un vehículo.

La asignación debe tener un estado válido.

El recorrido debe programarse de acuerdo con la programación de la microrruta.

No se debe asignar un vehículo que no esté disponible para realizar el recorrido.

No se debe asignar una cuadrilla que no esté disponible para la fecha programada.

7.**Relación con las historias de usuario**

HU10 Asignar recursos a una microrruta

Este endpoint permite cumplir los siguientes criterios:

Se debe poder seleccionar una microrruta.

Se debe poder seleccionar un vehículo disponible.

Se debe poder seleccionar una cuadrilla disponible.

El sistema debe detectar conflictos de asignación.

El sistema debe impedir completar la asignación cuando exista un conflicto.

Una asignación sin conflictos debe poder completarse correctamente.

HU11 Gestionar doble recorrido de vehículo de 7 yd^3

También puede contribuir a este requisito:

Permite registrar la asignación correspondiente al segundo recorrido, siempre que se implemente la lógica de planificación y asociación de ese recorrido.

8. Relación con la base de datos

Tablas involucradas:

assignment:

crew:

microroute:

vehicle:

assignment_status:

microroute_schedule:

relacionada:

relaciones :assignment.crew_id → crew.id

assignment.microroute_id → microroute.id

assignment.vehicle_id → vehicle.id

assignment.assignment_status_id → assignment_status.id

Endpoint 39/Modificar asignación

Método y ruta: PATCH /api/v1/assignments/{id}

Autenticación: Bearer Token (JWT).

2.¿Para qué sirve?

Permite modificar la información de una asignación existente, como la cuadrilla, la microrruta, el vehículo, el estado o la fecha programada, según las reglas de negocio establecidas.

parametro:

parámetro:

tipo:

obligatorio:

descripcion:

3.encabezado

Authorization: Bearer <access_token>

Content-Type: application/json

Accept: application/json

4. Cuerpo de la solicitud

Al ser un método PATCH, los campos son opcionales. Solo deben enviarse los datos que se desean modificar.


```
{
  "crewId": "uuid-nueva-cuadrilla",
  "microrouteId": "uuid-nueva-microrruta",
  "vehicleId": "uuid-nuevo-vehiculo",
  "assignmentStatusId": "uuid-nuevo-estado",
  "scheduledDate": "2026-11-06"
}
```


campo:crewId

tipo:UUID

obligatorio:no

descripción:Nueva cuadrilla asignada.

campo:microrouteId

tipo:UUID

obligatorio:no

descripción:Nueva microrruta.

campo:vehicleId

tipo:UUID

obligatorio:no

descripción: Nuevo vehículo asignado.

campo:assignmentStatusId

tipo:UUID

obligatorio:no

descripción:Nuevo estado de la asignación.

campo:scheduledDate

tipo:date

obligatorio:no

descripción:Nueva fecha programada.

5.respuesta exitosa 200 ok

{

"id": "uuid-asignacion",

"crewId": "uuid-nueva-cuadrilla",

"microrouteId": "uuid-microrruta",

"vehicleId": "uuid-nuevo-vehiculo",

"assignmentStatus": {

```
    "id": "uuid-estado",
    "code": "PENDING",
    "name": "Pendiente"
  },
  "scheduledDate": "2026-11-06",
  "startedAt": null,
  "endedAt": null,
  "createdAt": "2026-11-04T10:00:00Z",
  "updatedAt": "2026-11-04T11:00:00Z"
}
```


5.posibles errores

400 Bad Request: El identificador no tiene un formato UUID válido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar el historial.

404 Not Found: No existe una asignación con el identificador enviado.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

6. Validaciones

Verificar que el identificador de la asignación sea válido.

Comprobar que la asignación exista.

Verificar que se haya enviado al menos un campo para modificar.

Validar la existencia de los recursos que se desean cambiar.

Comprobar que los recursos estén activos.

Verificar que la fecha programada sea válida.

Validar que el cambio de estado corresponda a una transición permitida.

Comprobar que no se modifiquen datos que deban permanecer inalterables una vez iniciado o finalizado el recorrido.

7. Reglas de negocio

Solo se deben actualizar los campos enviados en la solicitud.

No se deben modificar asignaciones inexistentes.

Los cambios deben respetar la disponibilidad de los recursos.

Los cambios de estado deben respetar el ciclo de vida de la asignación.

Una asignación finalizada no debe modificarse libremente.

Las modificaciones deben quedar registradas para mantener la trazabilidad.

8. Relación con la base de datos

Tablas involucradas:

assignment:contiene la información de la asignación y permite actualizar sus datos, como las referencias a la cuadrilla, microrruta, vehículo y estado, así como la fecha programada.

crew:permite validar la cuadrilla seleccionada.

microroute:permite validar la microrruta asociada

vehicle:permite validar el vehículo seleccionado.

assignment_status:permite consultar y validar el nuevo estado de la asignación.

audit_log:almacena los registros de auditoría de las modificaciones realizadas

La actualización se realiza sobre assignment, utilizando las relaciones descritas anteriormente.

La trazabilidad de los cambios puede registrarse en audit_log, mediante los valores anteriores y nuevos de la asignación.

Endpoint 40/Consultar historial de una asignación

Método: GET

Ruta: /api/v1/assignments/{id}/history

Autenticación: Bearer Token

Body: No requiere.

2. Para qué sirve?

Permite consultar el historial de modificaciones realizadas sobre una asignación, incluyendo:

Qué información fue modificada.

Qué acción se realizó (creación, modificación o eliminación).

Quién realizó el cambio.

Cuándo ocurrió.

El historial lo registra el sistema de forma automática; este endpoint solo permite consultarlo.

Se utiliza para mantener la trazabilidad de los cambios realizados sobre una asignación.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El rol del usuario debe tener permisos para consultar el historial de cambios.

4. Encabezados HTTP

Authorization: Bearer <access_token>

Accept: application/json

5. Parámetros de ruta

No requiere cuerpo de solicitud.

Descripción de los campos:

Campo: id

Tipo: string (UUID)

Obligatorio: Sí.

Descripción: Identificador único de la asignación cuyo historial se desea consultar.

6. Respuesta exitosa

Código HTTP: 200 OK

Devuelve los cambios registrados, del más reciente al más antiguo. Si la asignación no tiene cambios registrados, history se devuelve vacío.

{

"assignmentId": "uuid-asignacion",

"history": [

```
  {
    "id": "uuid-registro-historial",
    "actor": "uuid-usuario",
    "action": "UPDATE",
    "oldValues": {
      "vehicle_id": "uuid-vehiculo-anterior",
      "assignment_status": "PENDING"
    },
    "newValues": {
      "vehicle_id": "uuid-vehiculo-nuevo",
      "assignment_status": "IN_PROGRESS"
    },
    "createdAt": "2026-11-04T11:00:00Z"
  }
 ]
}
```


7. Respuestas de error

400 Bad Request: El identificador no tiene un formato UUID válido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar el historial.

404 Not Found: No existe una asignación con el identificador enviado.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de devolver la información, el backend debe:

Validar la sesión y comprobar que el rol tenga permiso para consultar el historial.

Validar que el parámetro id tenga formato UUID.

Verificar que la asignación exista en la tabla assignment.

Consultar los registros de assignment_log filtrando por entity_id.

Ordenar los resultados del cambio más reciente al más antiguo.

Calcular newValues de cada registro según la regla de negocio.

Responder con código 404 si la asignación no existe.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar el historial de cambios.

El historial corresponde únicamente a la asignación solicitada.

Los registros del historial los genera el sistema de forma automática; no se crean ni se modifican desde este endpoint.

Cada registro identifica al responsable (actor): el identificador del usuario autenticado o SYSTEM cuando la acción fue automática.

assignment_log conserva los valores anteriores (old_values). Los valores nuevos (newValues) se calculan al consultar: corresponden a old_values del registro siguiente o, en el más reciente, al estado actual de la asignación.

La dirección IP (request_ip) se almacena en el registro, pero no se devuelve en esta respuesta.

10. Relación con la base de datos

Este endpoint involucra principalmente dos tablas:

assignment: permite verificar que la asignación exista y obtener su estado actual.

assignment_log: almacena el historial de la asignación: responsable (actor), acción (action: INSERT, UPDATE o DELETE), valores anteriores (old_values), dirección IP (request_ip) y fecha (created_at).

La relación principal es:

assignment_log.entity_id → assignment.id

11. Relación con las historias de usuario

HU12 Historial de cambios

Permite consultar cuándo se modificó una asignación y qué información cambió.

Permite identificar al responsable de cada modificación y conservar la trazabilidad de los cambios.

Endpoints independientes

Endpoint 41/Listar turnos de una microrruta

Método: GET

Ruta: /api/v1/microroutes/{id}/schedules

Autenticación: Bearer Token

Body: No requiere.

2. Para qué sirve?

Permite consultar los turnos programados de una microrruta, indicando en qué día de la semana opera cada uno.

Se utiliza para administrar los horarios de la microrruta y para verificar en qué turnos puede asignarse personal y vehículos.

Por defecto, el sistema devuelve únicamente las programaciones activas.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El rol del usuario debe tener permisos para consultar rutas.

4. Encabezados HTTP

Authorization: Bearer <access_token>

Accept: application/json

5. Parámetros de ruta y de consulta

No requiere cuerpo de solicitud.

Descripción de los campos:

Campo: id

Tipo: string (UUID)

Obligatorio: Sí.

Descripción: Identificador único de la microrruta cuyos turnos se desean consultar.

Campo: isActive

Tipo: boolean

Obligatorio: No.

Descripción: Filtra las programaciones activas o inactivas. Por defecto devuelve únicamente las activas.

Campo: dayOfWeek

Tipo: integer (1 a 7)

Obligatorio: No.

Descripción: Filtra por día de la semana (1 = lunes, 7 = domingo).

6. Respuesta exitosa

Código HTTP: 200 OK


```
[
 {
   "id": "uuid-microroute-schedule",
   "dayOfWeek": 1,
   "schedule": {
     "id": "uuid-turno",
     "name": "Turno mañana",
     "startTime": "06:00",
     "endTime": "14:00"
   },
   "isActive": true,
   "createdAt": "2026-11-04T08:00:00Z",
```

"updatedAt": "2026-11-04T08:00:00Z"
}
]

7. Respuestas de error

400 Bad Request: El identificador no tiene un formato UUID válido o algún parámetro de consulta tiene un valor inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para consultar rutas.

404 Not Found: No existe una microrruta con el identificador enviado.

500 Internal Server Error: Ocurrió un error inesperado durante la consulta.

8. Validaciones

Antes de devolver la información, el backend debe:

Validar la sesión y comprobar que el rol tenga permiso para consultar rutas.

Validar que el parámetro id tenga formato UUID.

Validar que isActive tenga un valor booleano válido cuando se proporcione.

Validar que dayOfWeek sea un entero entre 1 y 7 cuando se proporcione.

Verificar que la microrruta exista en la tabla microroute.

Consultar las programaciones en microroute_schedule, incluyendo el turno asociado desde schedule.

Aplicar los filtros recibidos. Si no se envía isActive, devolver solo las activas.

Ordenar los resultados por día de la semana y hora de inicio del turno.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden consultar rutas.

Por defecto, el sistema devuelve únicamente las programaciones activas.

Una microrruta inactiva puede consultarse, pero sus programaciones no deben usarse para nuevas asignaciones.

Una microrruta puede tener varias programaciones, en distintos días y turnos.

10. Relación con la base de datos

Este endpoint involucra principalmente tres tablas:

microroute: permite verificar que la microrruta exista.

microroute_schedule: contiene las programaciones de la microrruta y el día de la semana.

schedule: contiene el nombre y el horario de cada turno.

Las relaciones principales son:

microroute_schedule.microroute_id → microroute.id

microroute_schedule.schedule_id → schedule.id

11. Relación con las historias de usuario

HU8 Crear microrrutas

Permite consultar los turnos en los que opera la microrruta.

HU10 Asignar recursos a una microruta

Permite identificar las programaciones disponibles para crear asignaciones.

Endpoint 42/Asociar turno a una microrruta

Método: POST

Ruta: /api/v1/microroutes/{id}/schedules

Autenticación: Bearer Token

Body: Sí requiere.

2. Para qué sirve?

Permite programar una microrruta en un turno y un día de la semana.

La programación creada queda disponible para generar asignaciones de personal y vehículos.

Una microrruta puede tener varios turnos programados, en distintos días.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El rol del usuario debe tener permisos para administrar rutas.

4. Encabezados HTTP

Authorization: Bearer <access_token>

Content-Type: application/json

Accept: application/json

5. Parámetros de ruta y cuerpo de la solicitud

{

"scheduleId": "uuid-turno",

"dayOfWeek": 1

}

Descripción de los campos:

Campo: id (parámetro de ruta)

Tipo: string (UUID)

Obligatorio: Sí.

Descripción: Identificador único de la microrruta a la que se asocia el turno.

Campo: scheduleId

Tipo: string (UUID)

Obligatorio: Sí.

Descripción: Identificador del turno. Debe existir en la tabla schedule.

Campo: dayOfWeek

Tipo: integer (1 a 7)

Obligatorio: Sí.

Descripción: Día de la semana en que opera la microrruta en ese turno (1 = lunes, 7 = domingo).

6. Respuesta exitosa

Código HTTP: 201 Created

Devuelve la programación creada.


```
{
  "id": "uuid-microroute-schedule",
  "dayOfWeek": 1,
  "schedule": {
    "id": "uuid-turno",
    "name": "Turno mañana",
    "startTime": "06:00",
    "endTime": "14:00"
  },
  "isActive": true,
  "createdAt": "2026-11-04T08:00:00Z",
  "updatedAt": "2026-11-04T08:00:00Z"
}
```


7. Respuestas de error

400 Bad Request: Faltan campos obligatorios o los datos enviados no cumplen con el formato esperado.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para administrar rutas.

404 Not Found: No existe la microrruta o el turno enviado.

409 Conflict: La microrruta ya tiene programado ese turno en ese día, o la microrruta o el turno se encuentran inactivos.

500 Internal Server Error: Ocurrió un error inesperado durante el registro.

8. Validaciones

Antes de crear la programación, el backend debe:

Validar la sesión y comprobar que el rol tenga permiso para administrar rutas.

Validar que el parámetro id tenga formato UUID.

Verificar que se hayan enviado todos los campos obligatorios.

Validar que scheduleId tenga formato UUID.

Validar que dayOfWeek sea un entero entre 1 y 7.

Verificar que la microrruta exista y se encuentre activa.

Verificar que el turno exista y se encuentre activo en schedule.

Verificar que no exista otra programación con la misma microrruta, turno y día de la semana.

Crear el registro en microroute_schedule con is_active = true.

Registrar las fechas created_at y updated_at.

Registrar el cambio en microroute_log de forma automática, conservando en old_values los turnos programados antes del cambio.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden administrar rutas.

La misma combinación de microrruta, turno y día de la semana no puede repetirse.

Las programaciones nuevas se crean como activas.

El registro en el historial lo genera el sistema; no lo realiza el usuario.

Pendiente por definir: si una microrruta puede tener dos turnos distintos el mismo día. El modelo de datos lo permite.

Pendiente de modelo de datos: se recomienda agregar la restricción UNIQUE (microroute_id, schedule_id, day_of_week) en microroute_schedule para garantizar la regla anterior desde la base de datos.

Pendiente de modelo de datos: microroute_schedule no tiene tabla de historial propia, por lo que sus cambios se registran en microroute_log.

10. Relación con la base de datos

Este endpoint involucra principalmente cuatro tablas:

microroute: permite verificar que la microrruta exista y esté activa.

schedule: permite verificar que el turno exista y esté activo.

microroute_schedule: almacena la nueva programación.

microroute_log: almacena el registro automático del cambio.

Las relaciones principales son:

microroute_schedule.microroute_id → microroute.id

microroute_schedule.schedule_id → schedule.id

microroute_log.entity_id → microroute.id

11. Relación con las historias de usuario

HU8 Crear microrrutas

Permite definir los turnos en los que opera la microrruta.

HU10 Asignar recursos a una microruta

Las programaciones creadas son la base para generar asignaciones.

Endpoint 43/Actualizar turno programado de una microrruta

Método: PATCH

Ruta: /api/v1/microroutes/{id}/schedules/{microrouteScheduleId}

Autenticación: Bearer Token

Body: Sí requiere.

2. Para qué sirve?

Permite modificar el día de la semana de una programación existente o activarla y desactivarla.

La desactivación es lógica: el registro no se elimina, porque puede estar relacionado con asignaciones históricas.

Una programación inactiva no debe usarse para crear nuevas asignaciones.

Para cambiar el turno de una programación, se desactiva la existente y se crea una nueva.

3. Autenticación

Requiere una sesión activa y un Bearer Token válido.

El rol del usuario debe tener permisos para administrar rutas.

4. Encabezados HTTP

Authorization: Bearer <access_token>

Content-Type: application/json

Accept: application/json

5. Parámetros de ruta y cuerpo de la solicitud

Todos los campos del cuerpo son opcionales individualmente, pero debe enviarse al menos uno.

{

"dayOfWeek": 2,

"isActive": true

}

Descripción de los campos:

Campo: id (parámetro de ruta)

Tipo: string (UUID)

Obligatorio: Sí.

Descripción: Identificador único de la microrruta.

Campo: microrouteScheduleId (parámetro de ruta)

Tipo: string (UUID)

Obligatorio: Sí.

Descripción: Identificador único de la programación que se desea modificar.

Campo: dayOfWeek

Tipo: integer (1 a 7)

Obligatorio: No.

Descripción: Nuevo día de la semana (1 = lunes, 7 = domingo).

Campo: isActive

Tipo: boolean

Obligatorio: No.

Descripción: Indica si la programación se encuentra activa o inactiva.

6. Respuesta exitosa

Código HTTP: 200 OK

Devuelve la programación actualizada con la misma estructura del endpoint de listado de turnos.


```
{
  "id": "uuid-microroute-schedule",
  "dayOfWeek": 2,
  "schedule": {
    "id": "uuid-turno",
    "name": "Turno mañana",
    "startTime": "06:00",
    "endTime": "14:00"
  },
  "isActive": true,
  "createdAt": "2026-11-04T08:00:00Z",
  "updatedAt": "2026-11-05T09:00:00Z"
}
```


7. Respuestas de error

400 Bad Request: El cuerpo está vacío, contiene campos desconocidos o presenta datos con formato inválido.

401 Unauthorized: No se proporcionó un token válido, la sesión expiró o fue revocada.

403 Forbidden: El usuario no tiene permisos para administrar rutas.

404 Not Found: No existe la microrruta o la programación, o la programación no pertenece a esa microrruta.

409 Conflict: Ya existe otra programación con ese turno y día, o la programación no puede desactivarse porque tiene asignaciones pendientes o en curso, si esta restricción es aprobada.

500 Internal Server Error: Ocurrió un error inesperado durante el proceso.

8. Validaciones

Antes de actualizar la información, el backend debe:

Validar la sesión y comprobar que el rol tenga permiso para administrar rutas.

Validar que los parámetros id y microrouteScheduleId tengan formato UUID.

Verificar que la microrruta exista.

Verificar que la programación exista y pertenezca a la microrruta indicada.

Comprobar que se haya enviado al menos un campo en el cuerpo de la solicitud.

Rechazar los campos que no estén definidos para esta operación, como scheduleId.

Validar que dayOfWeek sea un entero entre 1 y 7 cuando se proporcione.

Verificar que el nuevo día no genere una programación duplicada para la misma microrruta y turno.

Validar que isActive tenga un valor booleano válido cuando se proporcione.

Verificar si la programación puede desactivarse cuando tenga asignaciones en estado PENDING o IN_PROGRESS en la tabla assignment, de acuerdo con la regla que se defina.

Actualizar updated_at con la fecha y hora de la modificación.

Registrar el cambio en microroute_log de forma automática, conservando en old_values el estado anterior de la programación.

9. Reglas de negocio

RN 01 Acceso según rol: solamente los usuarios autorizados pueden administrar rutas.

La programación no se elimina físicamente; se desactiva mediante el campo is_active.

Una programación inactiva no puede usarse en nuevas asignaciones, de acuerdo con HU10.

La información histórica se conserva para no afectar las asignaciones anteriores.

El registro en el historial lo genera el sistema; no lo realiza el usuario.

Pendiente por definir: qué ocurre con las asignaciones pendientes cuando se desactiva o se cambia el día de una programación.

Pendiente por definir: si se permite desactivar una programación con asignaciones en curso.

10. Relación con la base de datos

Este endpoint involucra principalmente cuatro tablas:

microroute: permite verificar que la microrruta exista.

microroute_schedule: permite actualizar day_of_week, is_active y updated_at.

assignment: permite verificar si la programación tiene asignaciones pendientes o en curso.

microroute_log: almacena el registro automático del cambio.

Las relaciones principales son:

microroute_schedule.microroute_id → microroute.id

assignment.microroute_schedule_id → microroute_schedule.id

microroute_log.entity_id → microroute.id

11. Relación con las historias de usuario

HU8 Crear microrrutas

Permite mantener actualizados los turnos en los que opera la microrruta.

HU10 Asignar recursos a una microruta

Una programación inactiva deja de estar disponible para crear nuevas asignaciones.